TUGAS AKHIR

IMPLEMENTASI WEBSITE WEB-BASED PURCHASE UNTUK SERVER MINECRAFT DENGAN PAYMENT GATEWAY MIDTRANS DAN INTEGRASI PTERODACTYL API

ditulis untuk memenuhi prasyarat pengambilan
mata kuliah Tugas Akhir

Oleh:

JULIUS SEBASTIAN WUWUNG
0706012210025



PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS CIPUTRA
SURABAYA

2026

UNIVERSITAS CIPUTRA
CREATING WORLD CLASS ENTREPRENEURS

PERNYATAAN KEASLIAN SKRIPSI / TUGAS AKHIR

Saya mahasiswa Program Studi S1 Informatika, Fakultas Teknologi Informasi, Universitas Ciputra,

Nama	: Julius Sebastian Wuwung
Nomor Induk Mahasiswa	: 0706012210025
Program Studi	: S1 Informatika

Dengan ini menyatakan bahwa, berkaitan dengan karya Tugas Akhir yang akan saya buat dengan judul: **"IMPLEMENTASI WEBSITE WEB-BASED PURCHASE UNTUK SERVER MINECRAFT DENGAN PAYMENT GATEWAY MIDTRANS DAN INTEGRASI PTERODACTYL API"**

adalah:

a. Dibuat dan diselesaikan sendiri, dengan menggunakan materi kuliah, tinjauan lapangan dan tinjauan pustaka serta jurnal acuan lainnya, seperti yang tertera dalam daftar pustaka pada Tugas Akhir saya.
b. Tugas Akhir yang saya buat ini, bukan merupakan karya duplikasi (baik sebagian maupun seluruhnya) dari karya tulis/karya desain lain yang sudah pernah dipublikasikan atau yang sudah pernah dipakai untuk mendapatkan gelar di Universitas lain, kecuali pada bagian-bagian sumber informasi yang dicantumkan dengan cara yang semestinya.
c. Karya Tugas Akhir yang saya buat ini, bukan merupakan karya terjemahan dari buku atau jurnal acuan yang tertera pada Tugas Akhir saya.

Apabila saya terbukti tidak memenuhi apa yang telah saya nyatakan di atas, maka Tugas Akhir saya ini dinyatakan **BATAL**.

Surabaya, 5 Januari 2026

Hormat saya pernyataan,



Julius Sebastian Wuwung

UNIVERSITAS CIPUTRA
CREATING WORLD CLASS ENTREPRENEURS

UNIVERSITAS CIPUTRA
FAKULTAS TEKNOLOGI INFORMASI

PERSETUJUAN DOSEN PEMBIMBING
TUGAS AKHIR

"IMPLEMENTASI WEBSITE WEB-BASED PURCHASE UNTUK SERVER MINECRAFT DENGAN PAYMENT GATEAWAY MIDTRANS DAN INTEGRASI PTERODACTYL API"

Oleh:

Nama	: Julius Sebastian Wuwung
Nomor Induk Mahasiswa	: 0706012210025
Program Studi	: S1 Informatika

Telah diperiksa dan disetujui untuk diajukan sebagai Tugas Akhir, guna mencapai gelar Sarjana Komputer pada Fakultas Teknologi Informasi, Universitas Ciputra.

Surabaya, 25 Mei 2026

Menyetujui,

Pembimbing

(Dr. Andreas Jodhinata, S.Kom., M.Kom.)

Ketua Program Studi

(Dr. Andreas Jodhinata, S.Kom., M.Kom.)

Dekan

(Prof. Dr. Adi Suryaputra P, S.Kom., M.Kom.)

UNIVERSITAS CIPUTRA
CREATING WORLD CLASS ENTREPRENEURS

UNIVERSITAS CIPUTRA
FAKULTAS TEKNOLOGI INFORMASI

PERSETUJUAN TIM PENGUJI TUGAS AKHIR

Pada hari Senin, 25 Mei 2026 telah diselenggarakan Sidang Tugas Akhir, untuk memenuhi gelar Sarjana Komputer pada Fakultas Teknologi Informasi Universitas Ciputra, atas nama:

Nama	: Julius Sebastian Wuwung
Nomor Induk Mahasiswa	: 0706012210025
Program Studi	: S1 Informatika

Dengan judul Tugas Akhir: **"IMPLEMENTASI WEBSITE WEB-BASED PURCHASE UNTUK SERVER MINECRAFT DENGAN PAYMENT GATEAWAY MIDTRANS DAN INTEGRASI PTERODACTYL API"**

Nama Tim Penguji	Tanda Tangan
1. Mychael Maoeretz Engel, S.Kom., M.Cs.	(Ketua Penguji)
2. Dr. Andreas Jodhinata, S.Kom., M.Kom.	(Anggota Penguji I)
3. Evan Tanuwijaya, S.Kom., M.Kom.	(Anggota Penguji II)

ABSTRAK

Server Minecraft yang dikelola oleh komunitas maupun penyedia layanan umumnya menyediakan berbagai produk digital seperti item, rank, dan mata uang virtual yang dapat dibeli oleh pemain. Namun, proses pembelian pada banyak server masih dilakukan secara manual melalui transfer dan konfirmasi kepada administrator, sehingga berpotensi menimbulkan keterlambatan verifikasi, kesalahan pencatatan transaksi, serta kurangnya transparansi riwayat pembelian. Oleh karena itu, penelitian ini bertujuan untuk mengembangkan website pembelian otomatis yang terintegrasi dengan sistem pembayaran online dan server Minecraft.

Metode yang digunakan dalam penelitian ini adalah metode Waterfall yang terdiri atas tahapan analisis kebutuhan, perancangan sistem, implementasi, pengujian, dan pemeliharaan. Sistem dikembangkan menggunakan framework Laravel sebagai backend, MySQL sebagai basis data, Midtrans Payment Gateway sebagai layanan pembayaran, serta Pterodactyl API untuk integrasi dengan server Minecraft. Pengujian sistem dilakukan menggunakan Unit Testing, Integration Testing, dan Beta Testing yang melibatkan satu administrator server dan empat pengguna sebagai responden.

Hasil penelitian menunjukkan bahwa sistem berhasil mengotomatisasi proses pembelian produk digital pada server Minecraft. Pengguna dapat melakukan login menggunakan username Minecraft dan email, memilih produk, melakukan pembayaran melalui Midtrans, serta menerima produk secara otomatis setelah pembayaran berhasil diverifikasi. Hasil pengujian menunjukkan bahwa seluruh 12 skenario Unit Testing dan 9 skenario Integration Testing memperoleh tingkat keberhasilan 100%. Selain itu, hasil Beta Testing menunjukkan bahwa sistem mampu mempercepat proses transaksi, meningkatkan transparansi riwayat pembelian, dan mengurangi kebutuhan intervensi manual dari administrator server.

Kontribusi utama penelitian ini adalah menyediakan solusi otomatisasi transaksi digital pada server Minecraft yang terintegrasi dengan sistem pembayaran online dan manajemen server. Sistem yang dikembangkan mampu meningkatkan efisiensi, kecepatan, dan keandalan proses pembelian bagi pengguna maupun administrator.

Kata Kunci : *Web-Based Purchase, Minecraft Server, Midtrans Payment Gateway, Pterodactyl API*, Sistem Transaksi Online

ABSTRACT

Minecraft servers managed by communities or service providers commonly offer various digital products, such as items, ranks, and virtual currencies that can be purchased by players. However, many servers still rely on manual purchasing processes involving bank transfers and separate payment confirmations to administrators. This approach often leads to delayed payment verification, transaction recording errors, and limited transparency regarding purchase histories. Therefore, this study aims to develop an automated web-based purchasing system integrated with online payment services and a Minecraft server environment.

The development process employed the Waterfall methodology, consisting of requirements analysis, system design, implementation, testing, and maintenance phases. The system was developed using the Laravel framework as the backend, MySQL as the database management system, Midtrans Payment Gateway for automated payment processing, and the Pterodactyl API for integration with the Minecraft server. System evaluation was conducted through Unit Testing, Integration Testing, and Beta Testing involving one server administrator and five active users as respondents.

The results of this study demonstrate that the developed system successfully automates the purchasing process of digital products on a Minecraft server. Users are able to log in using their Minecraft username and email address, browse available products, complete payments through Midtrans, and automatically receive purchased products after successful payment verification. The testing results indicate that all 12 Unit Testing scenarios and 9 Integration Testing scenarios achieved a 100% success rate. Furthermore, the Beta Testing results show that the system effectively accelerates transaction processing, improves purchase history transparency, and reduces the need for manual intervention by server administrators.

The primary contribution of this research is the development of an automated digital transaction solution for Minecraft servers that integrates online payment processing and server management. The proposed system enhances efficiency, transaction speed, reliability, and overall user experience for both players and server administrators.

Keywords : *Web-Based Purchase, Minecraft Server, Midtrans Payment Gateway, Pterodactyl API*, Online Transaction System

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga saya dapat menyelesaikan Tugas Akhir dengan judul **"IMPLEMENTASI WEBSITE WEB-BASED PURCHASE UNTUK SERVER MINECRAFT DENGAN PAYMENT GATEWAY MIDTRANS DAN INTEGRASI PTERODACTYL API"**. Tugas Akhir ini disusun untuk memenuhi sebagian persyaratan akademik guna memperoleh gelar Sarjana Komputer pada Program Studi Informatika, Fakultas Teknologi Informasi, Universitas Ciputra Surabaya.

Dalam penyusunan Tugas Akhir ini, saya menyadari bahwa keberhasilan ini tidak lepas dari bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, saya ingin menyampaikan ucapan terima kasih yang tulus kepada:

1. Keluarga tercinta, terutama kedua orang tua saya, yang senantiasa memberikan doa, dukungan moral, maupun materiil, serta kasih sayang yang tidak terhingga selama masa studi saya.
2. Bapak/Ibu Dosen Program Studi Informatika Universitas Ciputra, khususnya dosen pembimbing yang telah memberikan arahan, ilmu, dan bimbingan yang sangat berharga dalam penyempurnaan karya ini.
3. Seluruh civitas akademika Universitas Ciputra Surabaya yang telah menyediakan lingkungan belajar yang inovatif serta fasilitas yang mendukung perkembangan diri saya sebagai *full-stack developer*.
4. Sahabat-sahabat seperjuangan, terutama kepada **Jessen Jerome Richard Broto**, yang telah banyak membantu dalam proses pengerjaan, khususnya atas bantuan akses berbagai *plugin* Minecraft secara gratis yang menjadi komponen vital dalam keberhasilan integrasi sistem ini.
5. Teman-teman Informatika angkatan 2022 serta semua pihak yang telah memberikan semangat dan bantuan teknis maupun non-teknis yang tidak dapat saya sebutkan satu per satu.

Akhir kata, saya menyadari bahwa karya Tugas Akhir ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat saya harapkan. Semoga karya ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan, khususnya dalam bidang sistem transaksi digital dan manajemen komunitas *game online*.

Surabaya, 5 Januari 2026

Penulis
Julius Sebastian Wuwung

DAFTAR ISI

ABSTRAK iv
ABSTRACT v
KATA PENGANTAR vi
DAFTAR ISI viii
DAFTAR GAMBAR xi
DAFTAR TABEL xiii
DAFTAR SEGMEN xiv
DAFTAR LAMPIRAN xv

BAB 1 PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan 3
1.4 Manfaat 4
1.4.1 Manfaat Teoritis 4
1.4.2 Manfaat Praktis 4
1.5 Ruang Lingkup 4
1.6 Sistematika Penulisan 5

BAB 2 KAJIAN PUSTAKA 6
2.1 Tinjauan Pustaka 6
2.2 Landasan Teori 7
2.2.1 Teori Pendukung 7
2.2.2 Teknologi 8
2.2.2.1 Back-End: PHP dan Framework Laravel 8
2.2.2.2 Front-End: Bootstrap dan JavaScript 8
2.2.2.3 Application Programming Interface (API) 9

BAB 3 METODOLOGI PENELITIAN 10
3.1 Analisis Kebutuhan 10
3.1.1 Teknik Pengumpulan Data 10
3.1.2 Responden Penelitian 10
3.1.3 Instrumen Penelitian / Pertanyaan 10
3.1.4 Hasil yang Diperoleh 10

3.1.5 Spesifikasi Fitur 11
3.1.5.1 Fitur untuk Pengguna (Player Minecraft) 11
3.1.5.2 Fitur untuk Admin 11
3.1.6 Spesifikasi Perangkat Lunak (Software) 11
3.1.7 Spesifikasi Perangkat Keras (Hardware) dan Infrastruktur 12
3.2 Metode Pengembangan Software 12
3.3 Desain Sistem 13
3.3.1 Desain UML 13
3.3.1.1 Use Case Diagram 13
3.3.1.2 Use Case Scenario 14
3.3.1.3 Sequence Diagram 19
3.3.1.4 Class Diagram 26
3.3.2 Perancangan Database 27
3.3.3 Arsitektur Sistem 28
3.4 Desain User Interface (Wireframing) 29
3.4.1 Antarmuka Pengguna (Player / User Interface) 29
3.4.1.1 Wireframe Halaman Login (UC-01) 29
3.4.1.2 Wireframe Dashboard & Update Profile (UC-03) 30
3.4.1.3 Wireframe Katalog Web-Store (UC-04 & UC-05) 31
3.4.1.4 Wireframe Riwayat Transaksi User (UC-07) 33
3.4.2 Antarmuka Administrator (Admin Interface) 34
3.4.2.1 Wireframe Login Admin 34
3.4.2.2 Wireframe Riwayat Transaksi Global (UC-07) 34
3.4.2.3 Wireframe Provision Logs & Redeliver Tool (UC-08 & UC-09) 35

BAB 4 IMPLEMENTASI 36
4.1 Implementasi Sistem 36
4.1.1 UC-01 & UC-02: Login via Minecraft Name & Check Player Status 36
4.1.2 UC-03: Update Email 39
4.1.3 UC-04: View Products 41
4.1.4 UC-05: Purchase Product 43
4.1.5 UC-06: Update Credits 47
4.1.6 UC-07: View Purchase History 50
4.1.6.1 User View 50
4.1.6.2 Admin View 51
4.1.7 UC-08: View Provision Logs 52
4.1.8 UC-09: Redeliver Failed Items 53

BAB 5 PENGUJIAN DAN PEMBAHASAN 56
5.1 Alpha Testing 56
5.1.1 Unit Testing 56
5.1.2 Integration Testing 58
5.2 Beta Testing 60
5.2.1 Skema Pengujian 61
5.2.2 Instrumen Wawancara Ujicoba 62
5.3 Pembahasan 63
5.3.1 Analisis Keandalan Fungsional dan Kemudahan Pembayaran 63
5.3.2 Analisis Waktu Tanggap Pengiriman Koin (Latency) 64
5.3.3 Analisis Efisiensi Operasional Pengelola dan Fitur Unggulan 64
5.3.4 Evaluasi Antarmuka (UI/UX) dan Rekomendasi Pengembangan . . . 65
5.3.5 Temuan Bugs dan Tindak Lanjut 66

BAB 6 KESIMPULAN DAN SARAN 67
6.1 Kesimpulan 67
6.2 Saran 67

DAFTAR PUSTAKA 69

UNIVERSITAS CIPUTRA
ENTREPRENEURS & PROFESSIONALS

DAFTAR GAMBAR

Gambar 3.1 *use Case Diagram Sistem Web Jukiverse* 14
Gambar 3.2 *Sequence Diagram UC01 - Login via Minecraft Name* 20
Gambar 3.3 *Sequence Diagram UC02 - Check Player Status* 21
Gambar 3.4 *Sequence Diagram UC03 - Update Email* 22
Gambar 3.5 *Sequence Diagram UC04 - View Products* 23
Gambar 3.6 *Sequence Diagram UC05 - Purchase Product* 23
Gambar 3.7 *Sequence Diagram UC06 - Update Credits* 24
Gambar 3.8 *Sequence Diagram UC07 - View Purchase History* 25
Gambar 3.9 *Sequence Diagram UC08 - View Provision Logs* 25
Gambar 3.10 *Sequence Diagram UC09 - Redeliver Failed Items* 26
Gambar 3.11 *Class Diagram Sistem Jukiverse* 27
Gambar 3.12 *Entity Relationship Diagram Sistem Jukiverse* 28
gambar 3.13 *Desain arsitektur sistem Jukiverse Web System* 29
Gambar 3.14 *Wireframe Halaman Login (UC-01)* 30
Gambar 3.15 *Wireframe Dashboard User* 30
Gambar 3.16 *Wireframe Update Email (UC-03)* 31
Gambar 3.17 *Wireframe Katalog Produk (UC-04)* 32
Gambar 3.18 *Wireframe Checkout Produk (UC-05)* 33
Gambar 3.19 *Wireframe Riwayat Transaksi User Page (UC-07)* 33
Gambar 3.20 *Wireframe Login Admin* 34
Gambar 3.21 *Wireframe Riwayat Transaksi Admin Page (UC-07)* 35
Gambar 3.22 *Wireframe Panel Provision Logs dan Fitur Redeliver (UC-08 & UC-09)* 35
Gambar 4.1 *Antarmuka Halaman Utama Form Login Sistem Jukiverse.store* . . . 36
Gambar 4.2 *Tampilan Validasi Eror Saat Username Minecraft Tidak Terdaftar* .39
Gambar 4.3 *Tampilan Validasi Eror Saat Akun Terikat dengan Alamat Email Lain* 39
Gambar 4.4 *Antarmuka Dashboard Pengguna Setelah Autentikasi Berhasil* 39
Gambar 4.5 *Antarmuka Halaman Pengaturan Akun* 41
Gambar 4.6 *Tampilan Validasi dan Notifikasi Sukses Saat Mengganti Alamat Email* 41
Gambar 4.7 *Antarmuka Hasil Implementasi Katalog Produk Jukiverse.store* . . . 42
Gambar 4.8 *Antarmuka Halaman Konfirmasi Detail Pesanan Produk Toko Web* 44
Gambar 4.9 *Antarmuka Pop-up Snap Midtrans dengan Metode Pembayaran GoPay QRIS* 46

Gambar 4.10 *Tampilan Notifikasi Sistem Saat Transaksi Pembayaran Dinyatakan Berhasil* 47
Gambar 4.11 *Dokumentasi Hasil Otomatisasi Penambahan Saldo Koin pada Server Game* 49
Gambar 4.12 *Antarmuka Riwayat Transaksi - User View* 51
Gambar 4.13 *Antarmuka Log Transaksi - Admin View* 52
Gambar 4.14 *Antarmuka Panel Monitoring Log Provisioning Administrator* . . . 53
Gambar 4.15 *Antarmuka Log Transaksi Gagal dengan Opsi Tombol Retry* 55
Gambar 4.16 *Pembaruan Status Log Menjadi Success Setelah Dilakukan Intervensi Manual* 55
Gambar 4.17 *Output Konsol Pterodactyl Menunjukkan Eksekusi Command dari Intervensi Manual Berhasil* 55

CREATING WORLD CLASS
UC
UNIVERSITAS
CIPUTRA
ENTREPRENEURS & PROFESSIONALS

DAFTAR TABEL

Tabel 3.1 *Use Case Scenario - Login via Minecraft Name* 15
Tabel 3.2 *Use Case Scenario - Check Player Status* 15
Tabel 3.3 *Use Case Scenario - Update Email* 16
Tabel 3.4 *Use Case Scenario - View Products* 16
Tabel 3.5 *Use Case Scenario - Purchase Product* 17
Tabel 3.6 *Use Case Scenario - Update Credits* 17
Tabel 3.7 *Use Case Scenario - View Purchase History* 18
Tabel 3.8 *Use Case Scenario - View Provision Logs* 18
Tabel 3.9 *Use Case Scenario - Redeliver Failed Items* 19
Tabel 5.1 *Profil Penguji (Testers)* 57
Tabel 5.2 *Ringkasan Hasil Integration Testing Sistem Jukiverse* 59
Tabel 5.3 *Profil Penguji Beta Testing* 60
Tabel 5.4 *Pertanyaan Wawancara untuk Model Kualitas Produk* 61
Tabel 5.5 *Pertanyaan Wawancara untuk Model Kualitas Penggunaan* 62
Tabel 5.6 *Temuan Bugs dan Tindak Lanjut* 63

UNIVERSITAS CIPUTRA

DAFTAR SEGMEN

Segmen 4.1 *Implementasi Alur Login dan Verifikasi Player Remote* 38
Segmen 4.2 *Implementasi Pembaruan Data Profil Pengguna* 40
Segmen 4.3 *Implementasi Penampilan Katalog Produk* 42
Segmen 4.4 *Implementasi Validasi Produk dan Halaman Checkout* 44
Segmen 4.5 *Implementasi Pembuatan Transaksi Pembayaran Midtrans* 46
Segmen 4.6 *Implementasi Sinkronisasi Callback Midtrans* 48
Segmen 4.7 *Implementasi Otomatisasi Provisioning Koin* 49
Segmen 4.8 *Implementasi Riwayat Transaksi Pengguna* 50
Segmen 4.9 *Implementasi Log Transaksi Keseluruhan Administrator* 51
Segmen 4.10 *Implementasi Provision Logs Admin* 52
Segmen 4.11 *Implementasi Intervensi Manual Administrator* 55

UNIVERSITAS CIPUTRA

DAFTAR LAMPIRAN

LAMPIRAN A 70
LAMPIRAN B 71
LAMPIRAN C 75
LAMPIRAN D 79

CREATING WORLD CLASS ENTREPRENEURS & PROFESSIONALS
UC
UNIVERSITAS CIPUTRA

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan internet telah mendorong transformasi berbagai layanan digital, termasuk pada industri game online. Salah satu game yang memiliki komunitas aktif dan ekosistem layanan digital yang berkembang adalah Minecraft. Server Minecraft modern tidak hanya berfungsi sebagai media hiburan, tetapi juga berkembang menjadi platform ekonomi digital yang menyediakan berbagai layanan virtual seperti pembelian rank, item, mata uang virtual, dan paket langganan premium. Fenomena ini menunjukkan bahwa *virtual goods* memiliki nilai ekonomi nyata yang memengaruhi perilaku pengguna dalam komunitas digital (Lehdonvirta, 2009).

Pertumbuhan industri game online juga berdampak pada meningkatnya kebutuhan sistem transaksi digital yang cepat dan aman. Dalam konteks server Minecraft, transaksi digital menjadi bagian penting dalam mendukung keberlangsungan operasional server dan monetisasi layanan virtual. Oleh karena itu, diperlukan sistem pengelolaan transaksi yang efisien, aman, dan terstruktur agar kualitas layanan dapat tetap terjaga (Pressman, 2015).

Pada praktiknya, sebagian besar server Minecraft masih menggunakan mekanisme *top-up* manual. Pengguna biasanya melakukan pembayaran secara mandiri kemudian mengirimkan bukti transfer kepada administrator server untuk diverifikasi secara manual. Setelah pembayaran dinyatakan valid, administrator akan memberikan layanan atau item kepada pengguna melalui perintah server secara langsung. Proses ini sangat bergantung pada kecepatan respons admin sehingga sering menimbulkan keterlambatan layanan, kesalahan pencatatan transaksi, dan ketidakpastian bagi pengguna. Sistem manual juga berisiko menimbulkan *human error* pada proses validasi dan pengiriman layanan digital (Nurhayati & Setiawan, 2024).

Selain menimbulkan ketidakefisienan, sistem *top-up* manual memiliki keterbatasan dalam aspek skalabilitas. Semakin tinggi jumlah transaksi yang masuk, semakin besar pula beban kerja administrator server. Kondisi ini dapat menyebabkan penurunan kualitas layanan dan rendahnya tingkat kepuasan pengguna. Oleh

karena itu, diperlukan suatu sistem otomatisasi transaksi yang mampu melakukan validasi pembayaran dan pemberian layanan secara *real-time* tanpa ketergantungan penuh terhadap intervensi manusia. Otomatisasi layanan digital terbukti mampu meningkatkan efisiensi proses transaksi dan mengurangi risiko kesalahan operasional pada sistem berbasis web (Ramadhani et al., 2025).

Dalam pengembangan sistem transaksi digital modern, penggunaan *payment gateway* menjadi salah satu solusi utama untuk mendukung proses pembayaran otomatis. Midtrans merupakan salah satu layanan *payment gateway* yang menyediakan integrasi pembayaran digital melalui mekanisme Application Programming Interface (API). Melalui integrasi API dan *webhook*, sistem dapat melakukan validasi pembayaran secara otomatis segera setelah transaksi berhasil diproses oleh pihak penyedia layanan pembayaran. Implementasi *payment gateway* pada sistem berbasis web terbukti mampu meningkatkan efisiensi transaksi dan mempermudah proses verifikasi pembayaran digital (Nurhayati & Setiawan, 2024).

Konsep komunikasi antar sistem melalui API sendiri didukung oleh arsitektur REST (*Representational State Transfer*) yang memungkinkan pertukaran data secara fleksibel dan efisien melalui protokol HTTP. REST API menjadi standar umum dalam pengembangan layanan berbasis web modern karena mendukung interoperabilitas antar platform dan kemudahan integrasi sistem eksternal (Fielding, 2000).

Selain integrasi pembayaran, sistem juga membutuhkan mekanisme otomatisasi layanan pada server Minecraft. Salah satu panel manajemen server yang banyak digunakan adalah Pterodactyl Panel. Pterodactyl menyediakan REST API yang memungkinkan administrator mengontrol server secara terprogram, termasuk menjalankan *command*, memantau resource server, dan melakukan pengelolaan layanan secara otomatis (Everitt, 2016). Penggunaan REST API dalam manajemen layanan server dapat meningkatkan efisiensi pengelolaan resource dan mempercepat proses eksekusi layanan digital secara otomatis (Hamdana & Apriyani, 2020).

Pengembangan sistem top-up otomatis juga sejalan dengan tujuan Sustainable Development Goals (SDGs), khususnya SDG 9: *Industry, Innovation, and Infrastructure*, yang menekankan pentingnya pembangunan infrastruktur digital yang inovatif dan berkelanjutan. Implementasi otomatisasi transaksi digital dapat membantu meningkatkan efisiensi operasional serta mendukung transformasi layanan digital berbasis teknologi informasi.

Dalam proses pengembangan perangkat lunak, metode Waterfall merupakan salah satu metode yang banyak digunakan karena memiliki tahapan yang sistematis dan terstruktur mulai dari analisis kebutuhan hingga pengujian sistem. Pendekatan ini dinilai sesuai untuk pengembangan sistem dengan kebutuhan yang telah terdefinisi secara jelas sejak awal penelitian (Sasmito, 2017).

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan pembangunan sistem website *top-up* otomatis untuk server Minecraft dengan mengintegrasikan *payment gateway* Midtrans dan Pterodactyl API menggunakan framework Laravel. Laravel dipilih karena memiliki arsitektur Model-View-Controller (MVC) yang terstruktur dan mendukung integrasi API eksternal secara aman dan efisien (Stauffer, 2019). Sistem yang dikembangkan diharapkan mampu mengotomatisasi seluruh proses transaksi mulai dari pembayaran, validasi transaksi, pencatatan riwayat pembelian, hingga pemberian layanan secara *real-time* pada server Minecraft. Dengan adanya sistem ini, diharapkan kualitas layanan, efisiensi operasional, dan kepuasan pengguna dapat meningkat secara signifikan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun sistem website *top-up* otomatis untuk server Minecraft yang terintegrasi dengan payment gateway Midtrans dan Pterodactyl API?
2. Bagaimana mekanisme integrasi Midtrans API dan Pterodactyl API dalam mengotomatisasi proses pemberian layanan server Minecraft setelah pembayaran berhasil diverifikasi?
3. Bagaimana sistem dapat melakukan validasi transaksi pembayaran secara otomatis dan menangani kondisi kegagalan transaksi?

1.3 Tujuan

Tujuan dari penelitian ini adalah:

1. Merancang dan membangun sistem website *top-up* otomatis untuk server Minecraft berbasis web.

2. Mengintegrasikan payment gateway Midtrans dengan backend sistem untuk memvalidasi transaksi pembayaran secara otomatis.
3. Mengintegrasikan Pterodactyl API untuk melakukan otomatisasi pemberian layanan server Minecraft setelah pembayaran berhasil.
4. Membangun sistem monitoring transaksi dan pencatatan log aktivitas layanan secara terstruktur.
5. Menguji efektivitas dan efisiensi sistem dalam meningkatkan kualitas layanan transaksi digital pada server Minecraft.

1.4 Manfaat

1.4.1 Manfaat Teoritis

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan ilmu pengetahuan di bidang sistem informasi, khususnya terkait implementasi otomatisasi transaksi digital berbasis web menggunakan payment gateway dan REST API. Selain itu, penelitian ini dapat menjadi referensi akademik dalam pengembangan sistem layanan digital terintegrasi pada platform game online.

1.4.2 Manfaat Praktis

Secara praktis, sistem yang dibangun diharapkan mampu membantu administrator server Minecraft dalam mengelola transaksi secara otomatis dan efisien. Bagi pengguna, sistem ini memberikan kemudahan, transparansi, dan kecepatan layanan dalam proses pembelian layanan digital secara *real-time*.

1.5 Ruang Lingkup

Ruang lingkup penelitian ini meliputi:

1. Sistem difokuskan pada integrasi Midtrans API dan Pterodactyl API.
2. Sistem dikembangkan menggunakan framework Laravel dan database MySQL.
3. Pengujian difokuskan pada efektivitas otomatisasi transaksi dan pemberian layanan server Minecraft.
4. Penelitian tidak membahas keamanan jaringan server Minecraft secara mendalam.

1.6 Sistematika Penulisan

Sistematika penulisan laporan Tugas Akhir ini terdiri dari:

- **BAB 1 Pendahuluan**: Berisi latar belakang, rumusan masalah, tujuan, manfaat, ruang lingkup, dan sistematika penulisan.
- **BAB 2 Kajian Pustaka**: Membahas teori dan penelitian terdahulu yang mendukung penelitian.
- **BAB 3 Metodologi Penelitian**: Menjelaskan metode pengembangan sistem dan desain penelitian.
- **BAB 4 Implementasi**: Menjelaskan proses implementasi sistem dan integrasi API.
- **BAB 5 Pengujian dan Evaluasi**: Membahas hasil pengujian sistem dan evaluasi performa.
- **BAB 6 Kesimpulan dan Saran**: Berisi kesimpulan penelitian dan saran pengembangan sistem di masa mendatang.

UNIVERSITAS CIPUTRA

BAB 2
KAJIAN PUSTAKA

2.1 Tinjauan Pustaka

Tinjauan pustaka dilakukan untuk memetakan posisi penelitian ini di antara penelitian-penelitian terdahulu yang memiliki relevansi tema, khususnya dalam domain sistem transaksi digital dan otomatisasi manajemen server. Melalui perbandingan ini, dapat diidentifikasi celah inovasi yang akan diisi oleh sistem yang dikembangkan.

1. *Penelitian A (Nasional): "Implementasi Payment Gateway Midtrans Pada Aplikasi Toko Mas Berbasis Web" (2024) oleh Mila Nurhayati dan Ridwan Setiawan.*

   Penelitian ini membahas implementasi *payment gateway* Midtrans pada aplikasi toko emas berbasis web untuk mempermudah proses pembayaran digital secara *real-time*. Hasil penelitian menunjukkan bahwa integrasi Midtrans mampu membantu proses verifikasi pembayaran otomatis, mengurangi kesalahan pencatatan transaksi manual, serta meningkatkan kemudahan dan efisiensi transaksi pengguna pada sistem berbasis web (Nurhayati & Setiawan, 2024).

2. *Penelitian B (Nasional): "Perancangan Aplikasi E-Commerce Top-Up Game Online Berbasis Website dengan Integrasi Payment Gateway" (2025) oleh Dwi Arya Ramadhani, Purwa Esti Nugraha, M. Iqbal Syahrizal, dan Ramadhan Rakhmat Sani.*

   Penelitian ini berfokus pada pembangunan aplikasi *top-up* game online berbasis website dengan integrasi *payment gateway* untuk meningkatkan keamanan dan efisiensi transaksi digital. Sistem dikembangkan menggunakan metode *Waterfall* dan mampu mengotomatisasi proses pembayaran sehingga transaksi pembelian layanan game dapat dilakukan secara lebih cepat dan terstruktur (Ramadhani et al., 2025).

3. *Penelitian C (Nasional): "Rancang Bangun Sistem Informasi Top Up Game Online Berbasis Web dengan Framework Laravel (Studi Kasus Cheezy Store)" (2024) oleh Bramantio Alif Perdana.*

   Penelitian ini mengkaji pembangunan sistem informasi *top-up* game online berbasis web menggunakan framework Laravel. Penelitian menitikberatkan pada pengelolaan transaksi digital secara terintegrasi dengan antarmuka pengguna berbasis website. Hasil penelitian menunjukkan bahwa penggunaan framework Laravel

dapat membantu pengembangan sistem menjadi lebih terstruktur, efisien, serta mempermudah proses pengelolaan data transaksi pengguna (Perdana, 2024).

Berdasarkan ketiga tinjauan tersebut, penelitian ini menggabungkan konsep otomatisasi pembayaran digital menggunakan Midtrans dari Penelitian A, penerapan sistem *top-up* game online berbasis website dari Penelitian B, serta implementasi backend terstruktur menggunakan framework Laravel dari Penelitian C menjadi satu ekosistem terintegrasi untuk mengotomatisasi seluruh proses transaksi dan pengiriman layanan pada platform web-store Jukiverse.

2.2 Landasan Teori

2.2.1 Teori Pendukung

Ekonomi Digital dan Konsep Virtual Goods dalam Komunitas Game
Pertumbuhan ekosistem *game online* modern telah menggeser paradigma hiburan menjadi sebuah ruang ekonomi digital yang aktif. Fenomena ini didorong oleh konsumsi *Virtual Goods* (barang virtual), di mana pengguna bersedia mengalokasikan instrumen finansial riil untuk mendapatkan komoditas non-fisik di dalam *game*, seperti mata uang virtual (*virtual currency*), status keanggotaan (*rank*), ataupun item kosmetik.

Secara sosioteknis, transaksi ini didasari oleh prinsip *Value-Exchange Theory*, di mana nilai ekonomis ditukarkan dengan nilai fungsional, estetika, atau kepuasan psikologis di dalam komunitas virtual. Dalam membangun ekosistem ini, aspek *User Trust* (kepercayaan pengguna) menjadi parameter krusial. Transparansi data transaksi, kejelasan status pembayaran, dan konsistensi pencatatan log pada sistem basis data merupakan fondasi utama untuk memelihara retensi dan integritas ekonomi di dalam platform.

Otomatisasi Sistem dan Service Level Agreement (SLA)
Otomatisasi dalam arsitektur perangkat lunak didefinisikan sebagai transformasi proses manual terstruktur menjadi instruksi algoritmik terprogram guna meminimalkan intervensi manusia (*human intervention*). Penerapan otomatisasi pada sistem pemrosesan transaksi bertujuan untuk mengoptimalkan efisiensi operasional dan mereduksi potensi *human error* pada fase *data entry* atau penyerahan barang.

Dalam manajemen layanan TI, kecepatan pemrosesan data berkaitan erat dengan pemenuhan *Service Level Agreement* (SLA), yaitu standar kualitas layanan ekster-

nal yang dijanjikan kepada pengguna. Melalui mekanisme sinkronisasi data instan antara gerbang pembayaran (*payment gateway*) dan konsol server, pemenuhan SLA dapat dicapai dalam hitungan detik (*real-time provisioning*). Hal ini memberikan kepastian layanan kepada pengguna seketika setelah validasi finansial berhasil dilakukan oleh *backend* sistem.

Regulasi Komersial dan Kebijakan EULA Minecraft
Pengembangan platform pihak ketiga yang berinteraksi dengan ekosistem perangkat lunak komersial wajib tunduk pada batasan hukum yang ditetapkan oleh pemilik hak cipta. Dalam konteks ini, seluruh operasional toko web diatur oleh *Minecraft Commercial Usage Guidelines* dan *End User License Agreement* (EULA) yang diterbitkan oleh Mojang Synergies AB (Microsoft). Regulasi ini memberikan justifikasi hukum (*legal clearance*) bagi pemilik server untuk melakukan monetisasi luar jaringan (*out-of-game monetization*).

Kepatuhan terhadap EULA menitikberatkan pada asas *Fair Play* (keadilan bermain), di mana implementasi *virtual currency* (seperti koin toko) diizinkan selama bersifat *server-bound* (terisolasi hanya untuk jaringan server internal Jukiverse) dan tidak mengarah pada skema *Pay-to-Win* (P2W) yang merusak keseimbangan kompetitif antar pemain. Oleh karena itu, regulasi ini bertindak sebagai *business constraint* (batasan aturan bisnis) yang mengikat perancangan fitur, penentuan jenis produk pada database, hingga pembatasan hak akses perintah otomatis pada konsol server.

2.2.2 Teknologi

2.2.2.1 Back-End: PHP dan Framework Laravel

Pengembangan *back-end* menggunakan bahasa pemrograman PHP dengan framework Laravel. Pemilihan framework ini didasarkan pada kemampuannya dalam mendukung pengembangan aplikasi web yang terstruktur dan efisien. Laravel menyediakan berbagai fitur pendukung seperti sistem *routing*, *middleware*, *template engine* Blade, serta Guzzle HTTP Client yang digunakan untuk melakukan komunikasi dan permintaan (*request*) ke API eksternal secara aman.

2.2.2.2 Front-End: Bootstrap dan JavaScript

Bagian *front-end* built menggunakan HTML, CSS, dan JavaScript dengan bantuan *framework* Bootstrap. Penggunaan Bootstrap menjamin antarmuka yang responsif (*mobile-friendly*), mengingat tingginya mobilitas pengguna saat ini. JavaScript

digunakan untuk menangani logika pada sisi klien, seperti validasi formulir dan interaksi dinamis tanpa memuat ulang halaman (*Asynchronous*).

2.2.2.3 Application Programming Interface (API)

Sistem ini mengandalkan integrasi dua API utama:

- **Midtrans API:** Sebagai jembatan pembayaran yang menyediakan verifikasi otomatis melalui mekanisme *webhook*.
- **Pterodactyl API:** Sebagai pengontrol infrastruktur server yang memungkinkan eksekusi perintah konsol secara otomatis setelah status pembayaran berhasil.

CREATING WORLD CLASS ENTREPRENEURS & PROFESSIONALS
UC
UNIVERSITAS CIPUTRA

BAB 3
METODOLOGI PENELITIAN

3.1 Analisis Kebutuhan

Tahap awal ini bertujuan untuk menggali kebutuhan sistem berdasarkan permasalahan yang dihadapi calon pengguna. Pengumpulan data dilakukan melalui observasi terhadap proses bisnis lama yang kemudian digambarkan dalam bentuk *flowchart* untuk mengidentifikasi celah otomatisasi.

3.1.1 Teknik Pengumpulan Data

Data dikumpulkan melalui metode observasi langsung terhadap alur transaksi penjualan manual pada komunitas server game serta wawancara terstruktur dengan pengguna potensial maupun pengelola server terkait fungsionalitas yang dibutuhkan.

3.1.2 Responden Penelitian

Responden penelitian terdiri dari pengguna aktif (*player* game Minecraft) dan calon pengguna sistem, serta administrator server selaku pengelola utama manajemen eksekusi server game.

3.1.3 Instrumen Penelitian / Pertanyaan

Instrumen yang digunakan mencakup panduan wawancara dan lembar observasi aktivitas sistem. Pertanyaan difokuskan pada kendala efisiensi waktu konfirmasi pembayaran, hambatan pengiriman produk secara manual saat admin sedang tidak aktif, serta kendala pencatatan log riwayat pengiriman (*provision logs*).

3.1.4 Hasil yang Diperoleh

Hasil yang diperoleh menunjukkan kebutuhan mendesak akan sebuah platform webstore terintegrasi yang mampu melakukan verifikasi pembayaran otomatis (24 jam) serta langsung melakukan sinkronisasi penambahan koin/item secara *real-time* ke server game melalui automasi API tanpa intervensi manual dari pihak administrator.

3.1.5 Spesifikasi Fitur

Fitur sistem dikelompokkan berdasarkan peran pengguna (*user role*) sebagai berikut:

3.1.5.1 Fitur untuk Pengguna (Player Minecraft)

- *Login* menggunakan *username* Minecraft dan alamat email.
- Memperbarui alamat email profil yang tersimpan pada database website.
- Melihat daftar produk atau paket *top-up* yang tersedia.
- Melakukan pembelian dan pembayaran melalui *payment gateway* Midtrans.
- Melihat riwayat transaksi pribadi.

3.1.5.2 Fitur untuk Admin

- Memantau seluruh riwayat transaksi pengguna secara *real-time*.
- Melihat log aktivitas pengiriman item (*Provision Logs*) melalui Pterodactyl API.
- Melakukan pengiriman ulang item (*Redeliver*) jika terjadi kegagalan sistem pada transaksi tertentu.

3.1.6 Spesifikasi Perangkat Lunak (Software)

Perangkat lunak yang dibutuhkan dalam pengembangan dan pengoperasian sistem adalah sebagai berikut:

- **Sistem Operasi Server:** CloudLinux OS (dengan LiteSpeed Web Server pada infrastruktur Hostinger).
- **Bahasa Pemrograman dan Framework:** PHP (Laravel Framework) sebagai sistem utama *backend*, serta HTML, CSS, dan JavaScript (dengan Bootstrap) sebagai komponen *frontend*.
- **Database Management System:** MySQL untuk penyimpanan data pengguna, produk, dan riwayat transaksi secara terstruktur.
- **API Integrasi (Payment Gateway):** Midtrans API untuk otomatisasi dan menangani proses pembayaran digital secara *real-time*.
- **API Manajemen Server Minecraft (Automasi):** Pterodactyl API untuk eksekusi perintah otomatis, pembuatan akun, dan pengelolaan *resource* pada server game.
- **Lingkungan Pengujian (Browser):** Google Chrome untuk kebutuhan *debugging* dan pengujian antarmuka pengguna (*user interface*).

3.1.7 Spesifikasi Perangkat Keras (Hardware) dan Infrastruktur

Kebutuhan perangkat keras minimal dan infrastruktur jaringan yang digunakan dalam sistem ini adalah sebagai berikut:

- **Web Hosting:** Paket *Shared* / *Cloud Hosting* Hostinger dengan alokasi *resource* minimal 1 Core CPU dan 1 GB RAM guna menjaga stabilitas framework Laravel.
- **Server Minecraft (Node Target):** Ultra-server berbasis Linux dengan spesifikasi minimal 4 GB RAM, CPU *quad-core*, dan penyimpanan SSD untuk mengoperasikan instansi game server dan Pterodactyl Wings Daemon.
- **Koneksi Internet Server:** Koneksi internet yang stabil dengan kecepatan minimal 10 Mbps untuk memastikan kelancaran pertukaran data API eksternal (Midtrans Webhook dan Pterodactyl REST API).
- **Perangkat Pengembang dan Pengguna:** Komputer/Laptop dengan spesifikasi minimal RAM 8 GB (untuk kebutuhan *local development* menggunakan Composer dan Laragon/XAMPP) serta perangkat klien dengan spesifikasi standar untuk mengakses peramban web modern.

3.2 Metode Pengembangan Software

Metode pengembangan perangkat lunak yang digunakan dalam Tugas Akhir ini adalah metode *Waterfall*. Metode *Waterfall* dipilih karena memiliki tahapan yang terstruktur, sistematis, dan mudah dipahami, sehingga sesuai untuk pengembangan sistem dengan kebutuhan yang relatif jelas dan tidak sering mengalami perubahan selama proses pengembangan.

Metode *Waterfall* terdiri dari beberapa tahapan yang dilakukan secara berurutan, di mana setiap tahapan harus diselesaikan terlebih dahulu sebelum melanjutkan ke tahapan berikutnya, mulai dari analisis kebutuhan, desain sistem, pengodean (*coding*), pengujian (*testing*), hingga pemeliharaan (*maintenance*).

3.3 Desain Sistem

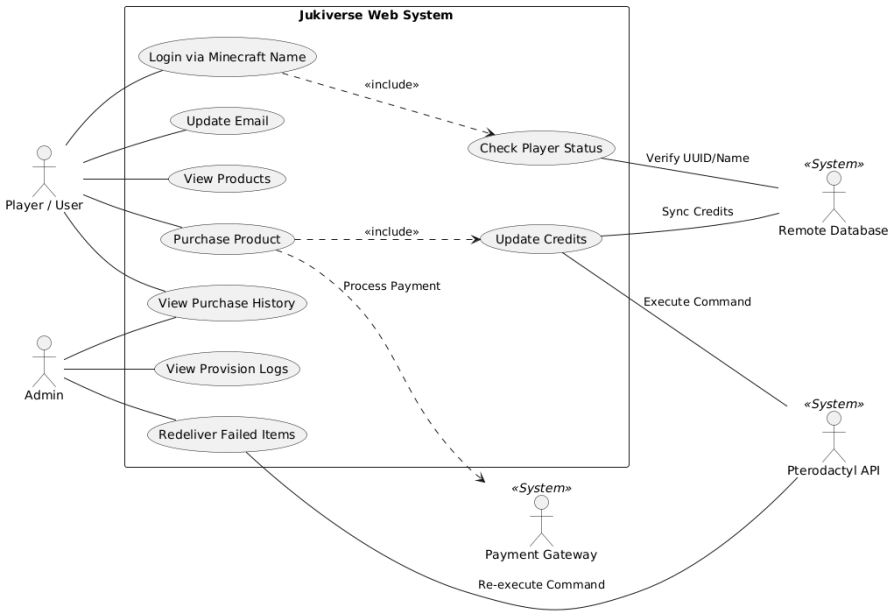
3.3.1 Desain UML

3.3.1.1 Use Case Diagram

Use case diagram dibuat menggunakan metode Unified Modeling Language (UML) untuk menggambarkan interaksi antara aktor (*Player/User* dan *Admin*) dengan sistem serta integrasi dengan API eksternal seperti Midtrans dan Pterodactyl. Diagram ini digunakan untuk menunjukan hak akses dan fungsi utama. Rincian interaksi ini dapat dilihat pada Gambar 3.1.

Pada use case diagram terdapat dua aktor utama yaitu *Player/User* yang berinteraksi dengan fitur login, update email, melihat produk, melakukan pembelian, dan melihat riwayat transaksi. Sedangkan aktor *Admin* memiliki akses untuk memantau riwayat transaksi global, melihat log pengiriman, dan melakukan pengiriman ulang item jika terjadi kegagalan.

Secara keseluruhan, use case diagram ini menggambarkan alur interaksi yang terjadi di dalam sistem Jukiverse, mulai dari proses login, pembelian produk, hingga pengelolaan transaksi dan log oleh administrator. Diagram ini menjadi dasar untuk pengembangan sistem yang terstruktur dan sesuai dengan kebutuhan pengguna.



Gambar 3.1: *Use Case Diagram Sistem Web Jukiverse*

3.3.1.2 Use Case Scenario

Berikut adalah detail skenario dari setiap *use case* yang dirancang pada subbab sebelumnya. Penjelasan detail mengenai alur masukan dan keluaran sistem dibagi menjadi sembilan skenario, yang masing-masing dijabarkan dari Tabel 3.1 hingga Tabel 3.9:

Table 3.1: *Use Case Scenario - Login via Minecraft Name*

Title	Login via Minecraft Name	UC01
Primary User	Player / User	
Pre-condition	User berada di halaman login website Jukiverse.	
Post-condition	User berhasil masuk ke dalam sistem dan diarahkan ke dashboard.	
Basic Flow	1. User memasukkan Minecraft Name. 2. Sistem memvalidasi nama ke API Mojang. 3. Sistem melakukan include Check Player Status (UC02). 4. User berhasil masuk ke dashboard.	

Table 3.2: *Use Case Scenario - Check Player Status*

Title	Check Player Status	UC02
Primary User	System	
Pre-condition	Terpanggil oleh UC01 untuk verifikasi pemain.	
Post-condition	Sistem berhasil memvalidasi status akun pemain.	
Basic Flow	1. Sistem mengirim data ke Remote Database Minecraft. 2. Remote Database mengonfirmasi validitas akun pemain.	

Table 3.3: *Use Case Scenario - Update Email*

Title	Update Email	UC03
Primary User	Player / User	
Pre-condition	User telah login dan berada di halaman pengaturan profil.	
Post-condition	Alamat email berhasil diperbarui pada database website.	
Basic Flow	1. User memasukkan alamat email baru. 2. Sistem memvalidasi format email. 3. Sistem memperbarui data email pada database internal website.	

Table 3.4: *Use Case Scenario - View Products*

Title	View Products	UC04
Primary User	Player / User	
Pre-condition	User mengakses halaman store.	
Post-condition	Daftar produk aktif berhasil ditampilkan kepada user.	
Basic Flow	1. Sistem mengambil daftar produk aktif dari database. 2. Sistem menampilkan nama, harga, dan deskripsi produk.	

Table 3.5: *Use Case Scenario - Purchase Product*

Title	Purchase Product	UC05
Primary User	Player / User	
Pre-condition	User memilih salah satu produk atau paket coin.	
Post-condition	Transaksi pembayaran berhasil dibuat pada sistem.	
Basic Flow	1. User menekan tombol pembelian. 2. Sistem memproses integrasi Midtrans. 3. Sistem membuat transaksi pembayaran. 4. Jika pembayaran berhasil maka sistem menjalankan Update Credits (UC06).	

Table 3.6: *Use Case Scenario - Update Credits*

Title	Update Credits	UC06
Primary User	System	
Pre-condition	Sistem menerima notifikasi pembayaran sukses dari Midtrans.	
Post-condition	Coin berhasil ditambahkan ke akun pemain di server Minecraft.	
Basic Flow	1. Sistem memperbarui status transaksi. 2. Sistem memperbarui jumlah coin pemain. 3. Sistem mengirim command ke Pterodactyl API. 4. Server Minecraft menjalankan command pemberian coin.	

Table 3.7: *Use Case Scenario - View Purchase History*

Title	View Purchase History	UC07
Primary User	Player / User dan Admin	
Pre-condition	User atau admin membuka halaman riwayat transaksi.	
Post-condition	Riwayat transaksi berhasil ditampilkan.	
Basic Flow	1. Sistem mengambil data transaksi dari database. 2. Sistem menampilkan status transaksi dan detail pembelian.	

Table 3.8: *Use Case Scenario - View Provision Logs*

Title	View Provision Logs	UC08
Primary User	Admin	
Pre-condition	Admin membuka halaman provision logs.	
Post-condition	Log pengiriman coin berhasil ditampilkan.	
Basic Flow	1. Sistem mengambil data provision logs dari database. 2. Sistem menampilkan status pengiriman dan pesan log.	

Table 3.9: *Use Case Scenario - Redeliver Failed Items*

Title	Redeliver Failed Items	UC09
Primary User	Admin	
Pre-condition	Terdapat transaksi gagal pada provision logs.	
Post-condition	Command pengiriman coin berhasil dikirim ulang ke server Minecraft.	
Basic Flow	1. Admin menekan tombol retry provision. 2. Sistem mengambil data transaksi gagal. 3. Sistem mengirim ulang command ke Pterodactyl API. 4. Sistem memperbarui status provision logs.	

3.3.1.3 Sequence Diagram

Sequence diagram digunakan untuk memodelkan interaksi dinamis antar objek dalam sistem berdasarkan skenario yang terdapat pada setiap *use case* yang telah dirancang sebelumnya. Diagram ini menggambarkan urutan pesan (*message*) yang dikirimkan antar entitas, baik yang berasal dari faktor eksternal seperti Player/User dan Administrator, maupun komponen internal sistem seperti Laravel Controller, Database MySQL, Midtrans Payment Gateway, Mojang API, dan Pterodactyl API.

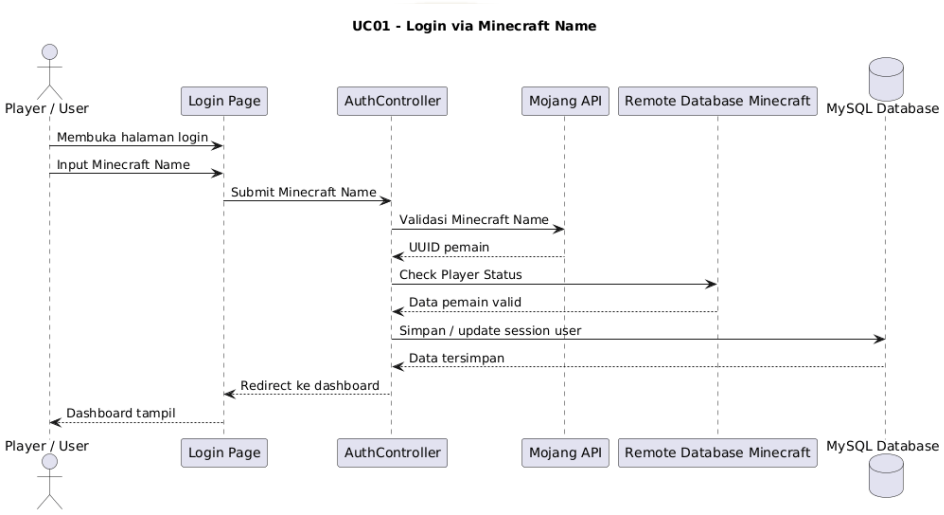
Melalui sequence diagram ini, dapat terlihat bagaimana setiap permintaan dari pengguna diproses oleh sistem hingga menghasilkan respons yang sesuai. Diagram ini juga membantu dalam memahami alur komunikasi antar modul secara terstruktur dan berurutan sehingga mempermudah proses implementasi dan pengembangan sistem.

Selain itu, sequence diagram berperan penting dalam memastikan bahwa setiap skenario *use case* telah memiliki alur proses yang jelas dan konsisten dengan arsitektur sistem *monolith* berbasis Laravel yang digunakan pada penelitian ini.

Berikut merupakan sequence diagram untuk masing-masing skenario *use case* yang terdapat pada sistem web store Minecraft Jukiverse.

3.3.1.4 UC01 - Login via Minecraft Name

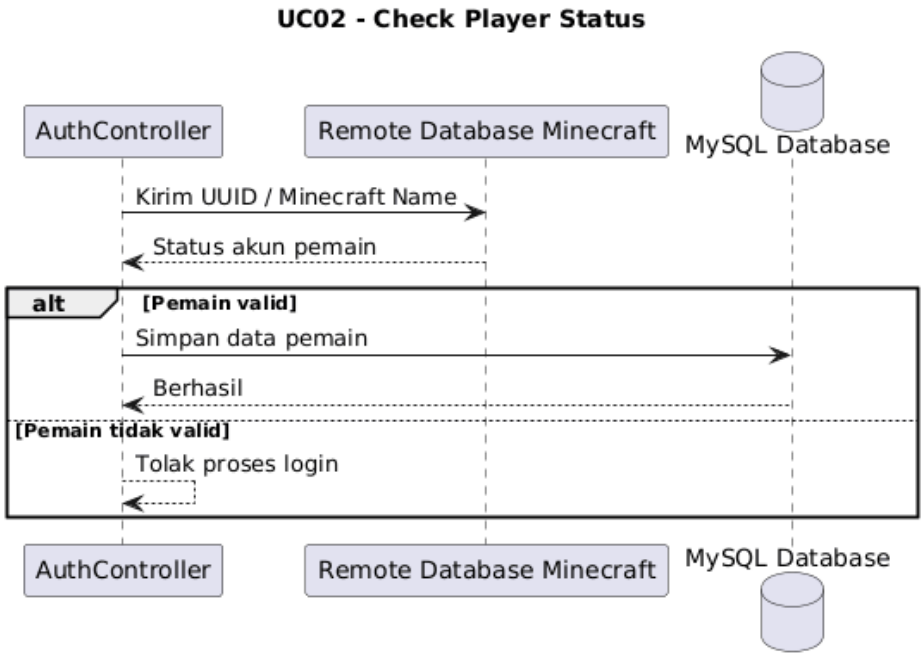
Sequence diagram ini menggambarkan proses autentikasi pengguna menggunakan *Minecraft Name*, yang visualisasinya ditunjukkan pada Gambar 3.2. Pengguna memasukkan nama akun Minecraft melalui halaman login, kemudian sistem melakukan validasi akun ke Mojang API untuk memperoleh UUID pemain. Setelah validasi berhasil, sistem melakukan pengecekan status akun pemain pada database server Minecraft sebelum membuat session login dan mengarahkan pengguna ke dashboard utama.



Gambar 3.2: *Sequence Diagram UC01 - Login via Minecraft Name*

3.3.1.5 UC02 - Check Player Status

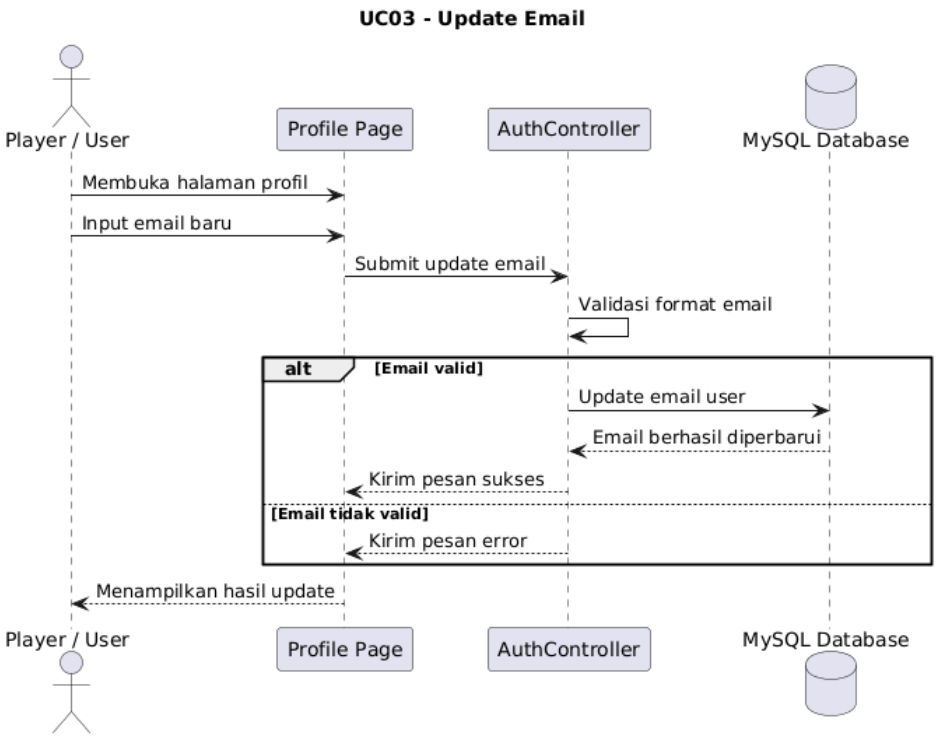
Diagram ini menunjukkan proses pengecekan status akun pemain pada database server Minecraft yang diilustrasikan pada Gambar 3.3. Sistem mengirimkan data UUID atau nama pemain ke remote database untuk memastikan bahwa akun pemain valid dan terdaftar pada server Minecraft sebelum proses login dilanjutkan.



Gambar 3.3: *Sequence Diagram UC02 - Check Player Status*

3.3.1.6 UC03 - Update Email

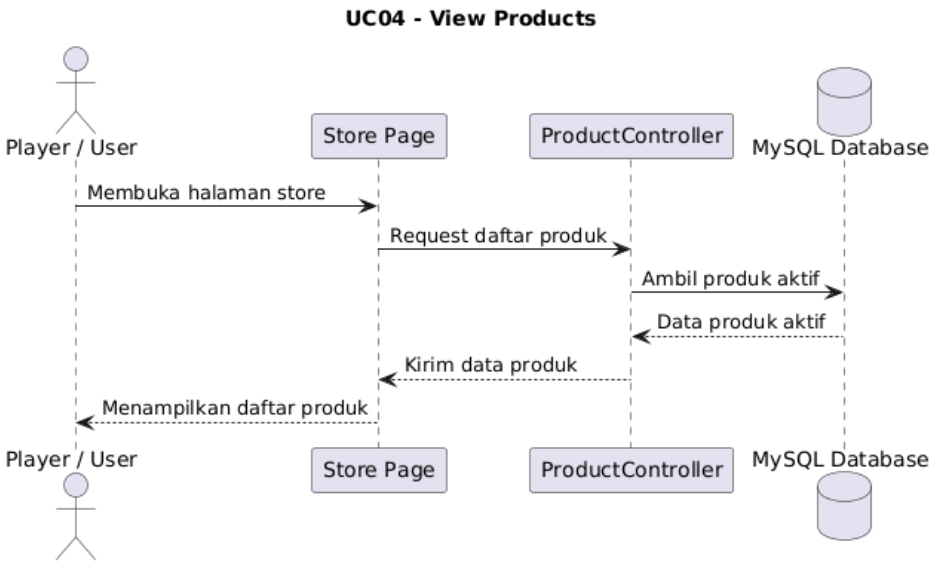
Sequence diagram pada Gambar 3.4 menggambarkan proses perubahan alamat email pengguna pada sistem website. Pengguna memasukkan alamat email baru melalui halaman profil, kemudian sistem melakukan validasi format email sebelum memperbarui data email pada database internal website.



Gambar 3.4: *Sequence Diagram UC03 - Update Email*

3.3.1.7 UC04 - View Products

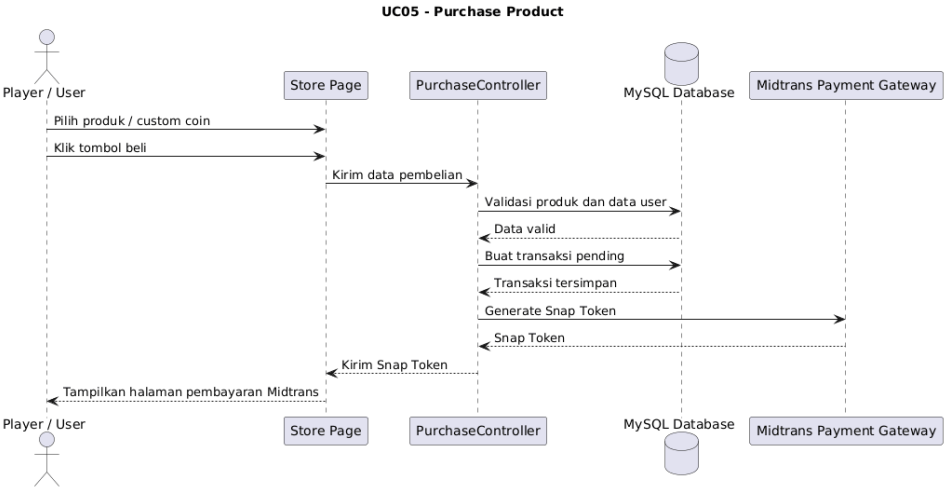
Diagram pada Gambar 3.5 menjelaskan proses pengambilan dan penampilan data produk pada halaman store. Sistem mengambil daftar produk aktif dari database kemudian menampilkannya kepada pengguna dalam bentuk katalog produk yang berisi nama produk, harga, dan deskripsi produk.



Gambar 3.5: *Sequence Diagram UC04 - View Products*

3.3.1.8 UC05 - Purchase Product

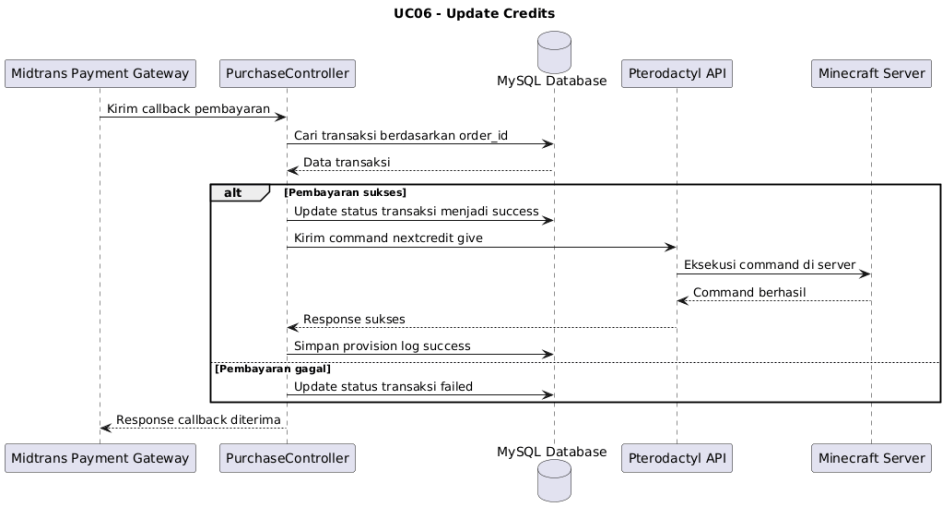
Sequence diagram yang tertera pada Gambar 3.6 menggambarkan proses pembelian produk atau paket coin pada sistem web store Minecraft. Pengguna memilih produk dan melakukan checkout, kemudian sistem membuat transaksi pembayaran dan melakukan integrasi dengan Midtrans Payment Gateway untuk menghasilkan *Snap Token* yang digunakan pada proses pembayaran digital.



Gambar 3.6: *Sequence Diagram UC05 - Purchase Product*

3.3.1.9 UC06 - Update Credits

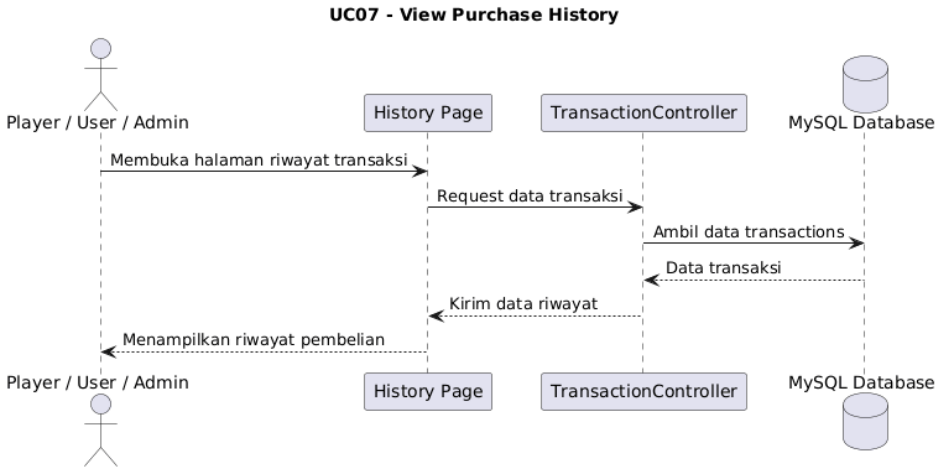
Diagram pada Gambar 3.7 menggambarkan proses sinkronisasi pembayaran dan pengiriman coin ke server Minecraft. Setelah sistem menerima callback pembayaran berhasil dari Midtrans, status transaksi diperbarui menjadi *success*. Sistem kemudian mengirim command ke Pterodactyl API untuk dieksekusi pada server Minecraft sehingga coin berhasil diberikan kepada pemain.



Gambar 3.7: *Sequence Diagram UC06 - Update Credits*

3.3.1.10 UC07 - View Purchase History

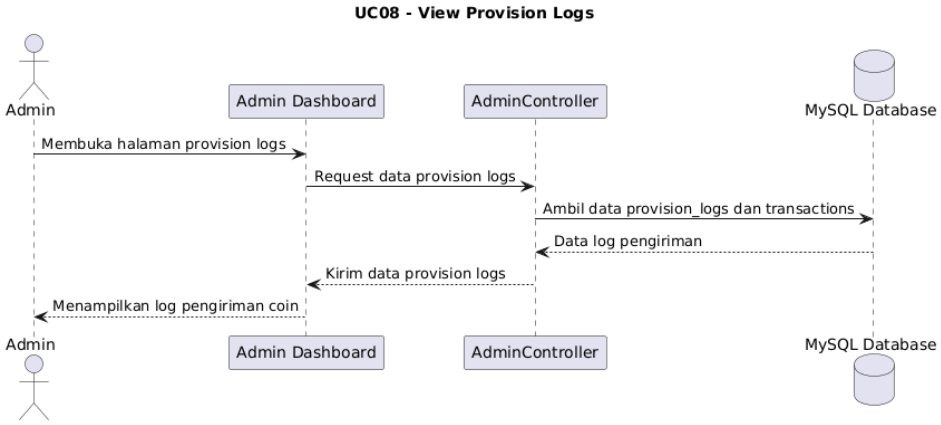
Sequence diagram pada Gambar 3.8 menunjukkan proses pengguna maupun administrator dalam melihat riwayat transaksi pembelian. Sistem mengambil data transaksi dari database kemudian menampilkan detail pembelian beserta status pembayaran kepada pengguna melalui halaman riwayat transaksi.



Gambar 3.8: *Sequence Diagram UC07 - View Purchase History*

3.3.1.11 UC08 - View Provision Logs

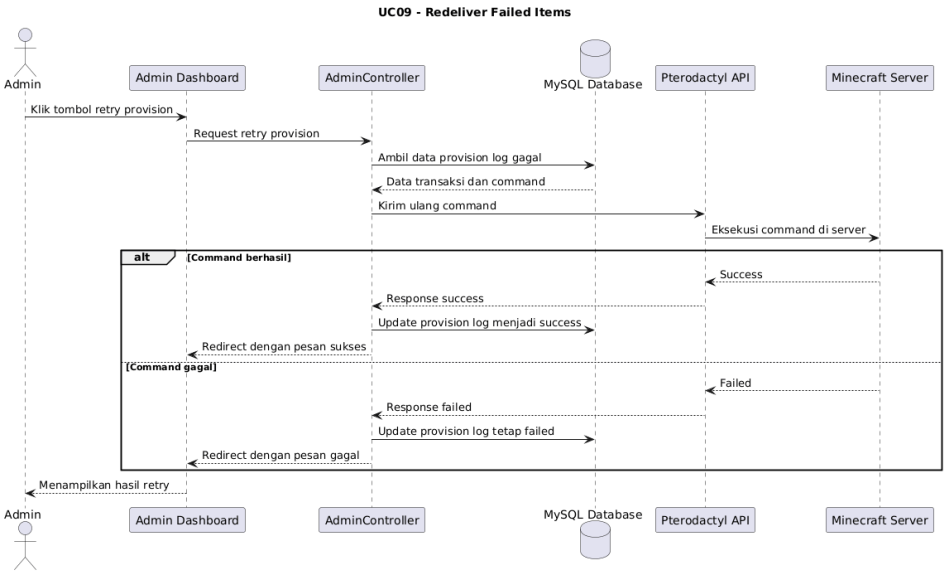
Diagram pada Gambar 3.9 menggambarkan proses pengambilan data *provision logs* oleh administrator. Sistem mengambil data log pengiriman coin atau item dari database kemudian menampilkannya pada halaman *Provision Logs* untuk mempermudah monitoring status pengiriman.



Gambar 3.9: *Sequence Diagram UC08 - View Provision Logs*

3.3.1.12 UC09 - Redeliver Failed Items

Sequence diagram pada Gambar 3.10 menunjukkan proses pengiriman ulang transaksi gagal pada sistem. Administrator memilih transaksi gagal pada halaman *Provision Logs* kemudian menekan tombol *Retry Provision*. Sistem mengirim ulang command ke Pterodactyl API dan memperbarui status log berdasarkan hasil pengiriman ulang tersebut.

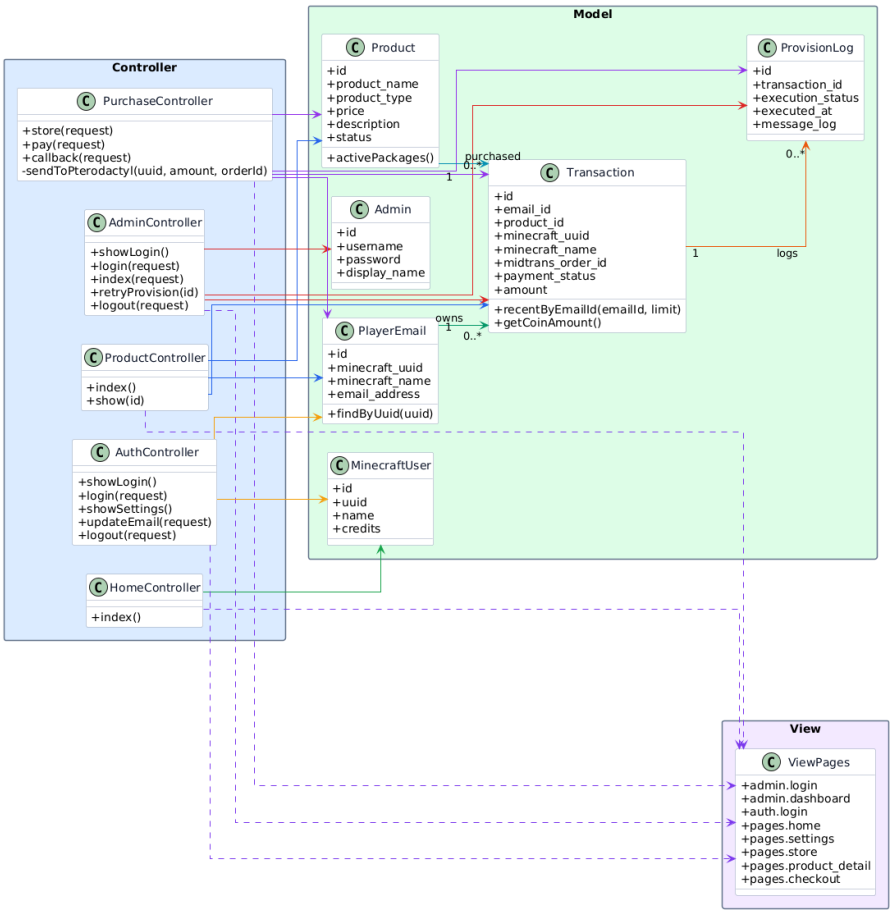


Gambar 3.10: *Sequence Diagram UC09 - Redeliver Failed Items*

3.3.2 Class Diagram

Struktur kelas, properti, metode, serta hubungan antar-objek logika penyusun sistem dimodelkan secara komprehensif melalui Class Diagram yang ditunjukkan pada Gambar 3.11. Diagram ini menjadi acuan utama dalam pembuatan file *Controller* dan *Model* pada repositori Laravel.

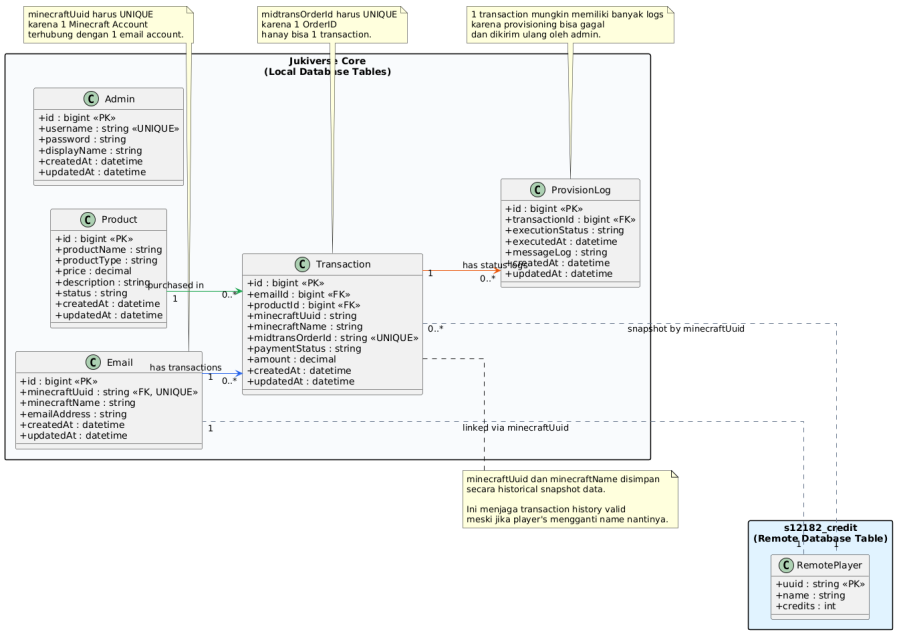
UNIVERSITAS CIPUTRA



Gambar 3.11: *Class Diagram Sistem Jukiverse*

3.3.3 Perancangan Database

Perancangan database ditampilkan menggunakan *Entity Relationship Diagram* (ERD) untuk menjelaskan pemetaan entitas dan relasi tabel seperti *users*, *products*, *transactions*, *provision_logs* dan *remote database user*. Detail rancangan arsitektur data internal ini digambarkan secara struktural pada Gambar 3.12.

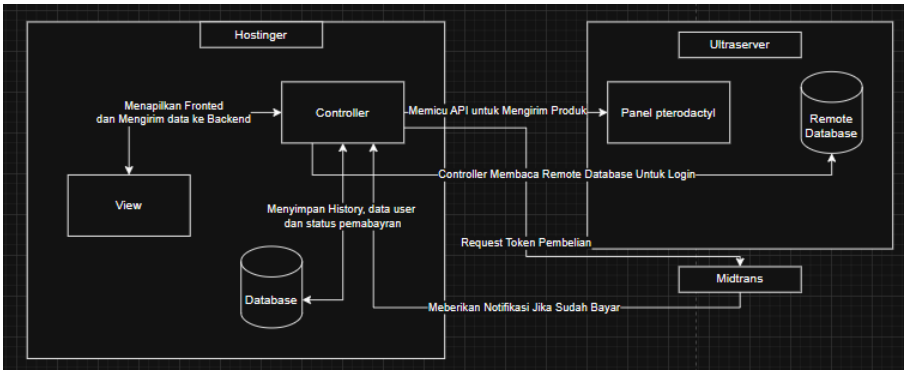


Gambar 3.12: *Entity Relationship Diagram Sistem Jukiverse*

3.3.4 Arsitektur Sistem

Sistem ini menggunakan arsitektur *Monolith* terintegrasi. Inti aplikasi dibangun menggunakan kerangka kerja Laravel *Full-Stack* yang berjalan di lingkungan */server CloudLinux* dan *LiteSpeed Web Server* milik Hostinger.

Seluruh berkas visual *frontend* berupa Blade template (*.blade.php*) dan logika pemrograman *backend* (PHP) terpasang di satu infrastruktur server yang sama. Aplikasi kemudian menjalin komunikasi aman dengan pihak ketiga menggunakan REST API secara sinkron dan asinkron melalui protokol HTTPS untuk memicu *Payment Gateway* Midtrans dan kontrol panel Pterodactyl Server Game. Rancangan topologi infrastruktur ini dipetakan secara detail pada Gambar 3.13.



Gambar 3.13: *Desain arsitektur sistem Jukiverse Web System*

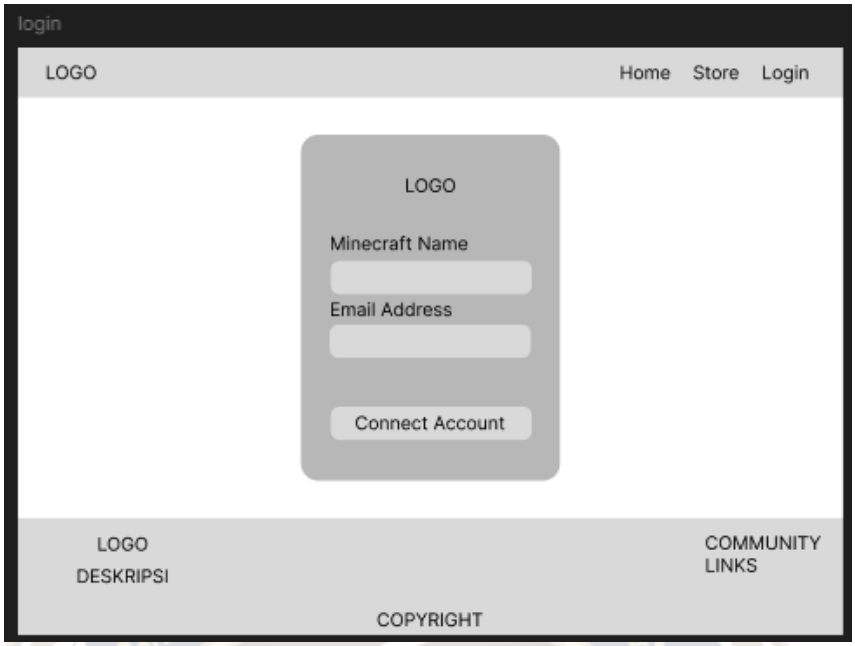
3.4 Desain User Interface (Wireframing)

Desain antarmuka aplikasi (*wireframe*) ini dirancang secara terukur berdasarkan kebutuhan pengguna yang telah dijabarkan pada skenario *use case* sebelumnya. Berikut adalah visualisasi rancangan layar aplikasi beserta keterkaitan kode fungsinya:

3.4.1 Antarmuka Pengguna (Player / User Interface)

3.4.1.1 Wireframe Halaman Login (UC-01)

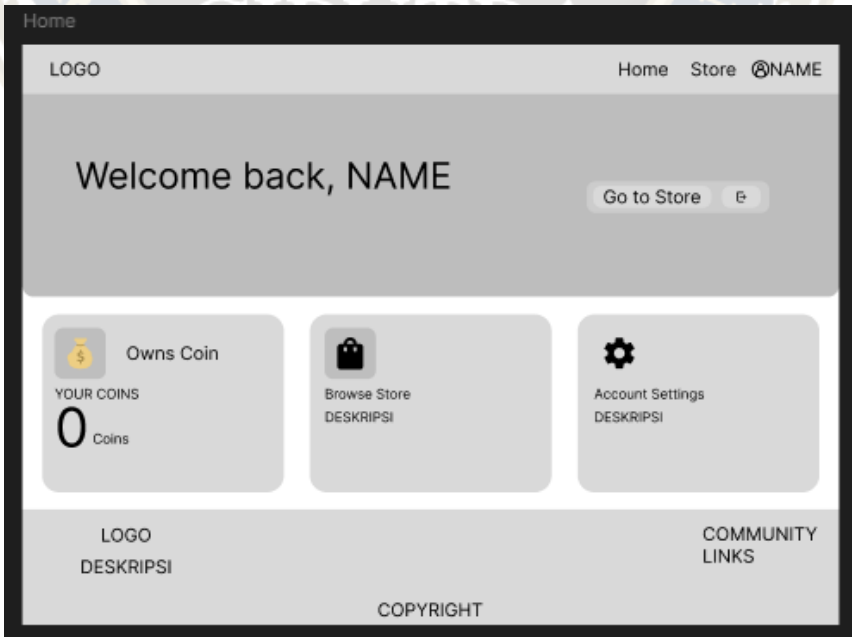
Antarmuka ini memfasilitasi pengguna untuk masuk ke dalam aplikasi menggunakan nama akun game terdaftar mereka. Tata letak elemen masukan halaman ini dapat dilihat pada Gambar 3.14.



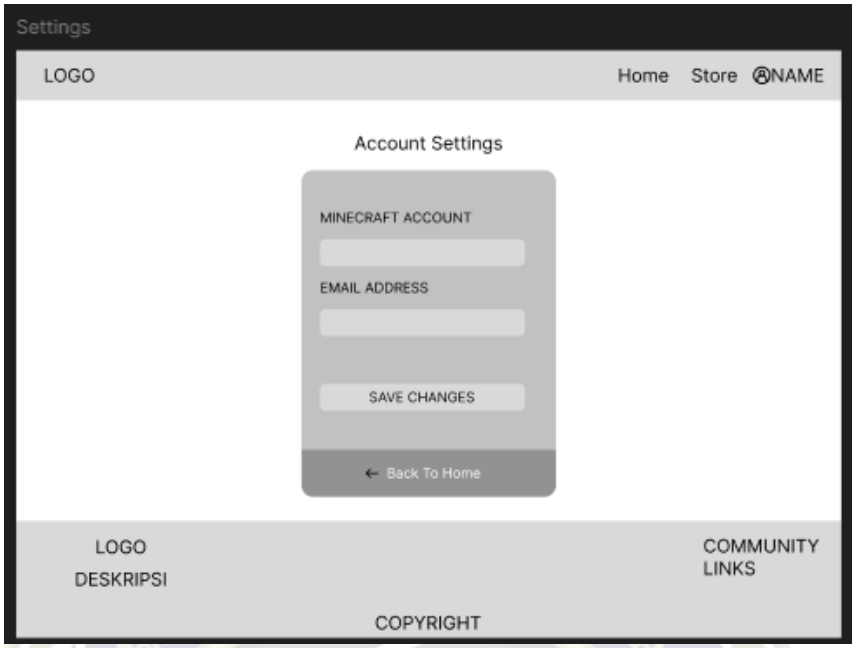
Gambar 3.14: *Wireframe Halaman Login (UC-01)*

3.4.1.2 Wireframe Dashboard & Update Profile (UC-03)

Halaman dashboard user yang memuat gambaran umum profil pemain ditunjukkan pada Gambar 3.15. Lebih lanjut, formulir khusus bagi pengguna yang ingin memperbarui data alamat email pada basis data internal disajikan secara terperinci pada Gambar 3.16.



Gambar 3.15: *Wireframe Dashboard User*

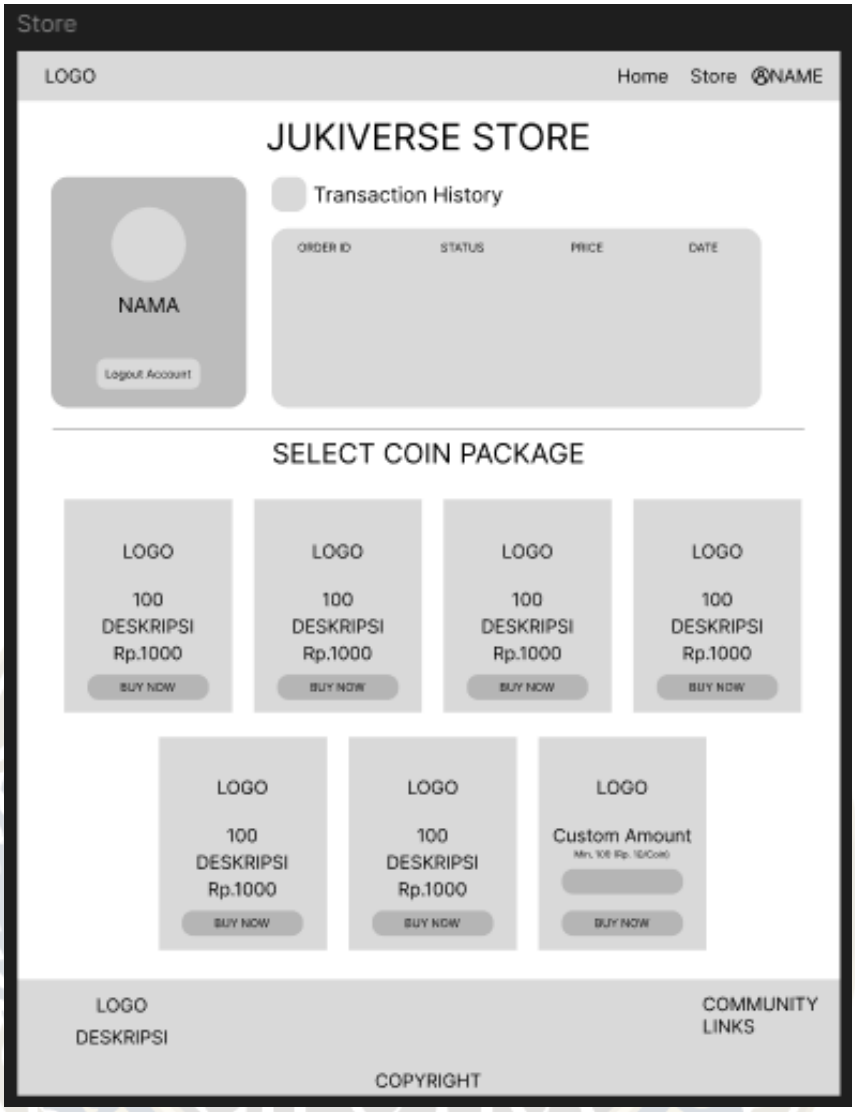


Gambar 3.16: *Wireframe Update Email (UC-03)*

3.4.1.3 Wireframe Katalog Web-Store (UC-04 & UC-05)

Layar katalog yang menyajikan visualisasi ragam produk aktif dari server game ditunjukkan pada Gambar 3.17. Sementara itu, tampilan lembar konfirmasi pemesanan beserta tombol aksi pembayaran digital otomatis (*checkout*) ditunjukkan pada Gambar 3.18.

UNIVERSITAS CIPUTRA
ENTREPRENEURS & PROFESSIONALS



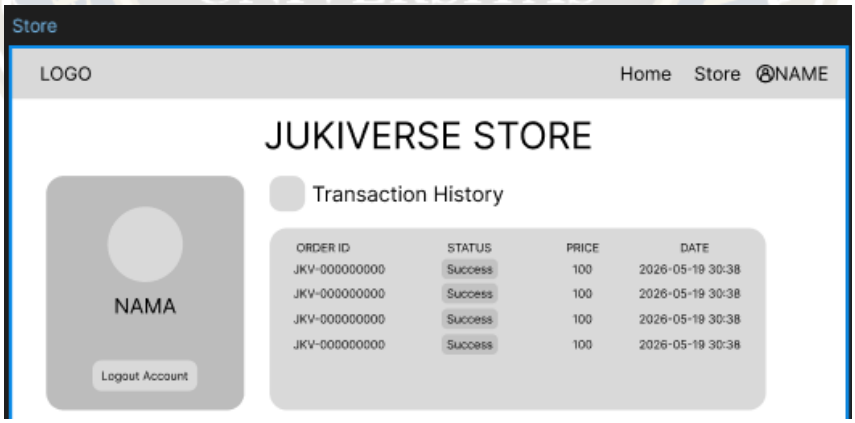
Gambar 3.17: *Wireframe Katalog Produk (UC-04)*



Gambar 3.18: *Wireframe Checkout Produk (UC-05)*

3.4.1.4 Wireframe Riwayat Transaksi User (UC-07)

Halaman ini menampilkan riwayat pembelian produk perseorangan yang telah dilakukan oleh pengguna, lengkap dengan status pembayaran dan detail nomor *invoice*. Visualisasi rancangan layout halaman ini ditunjukkan pada Gambar 3.19.

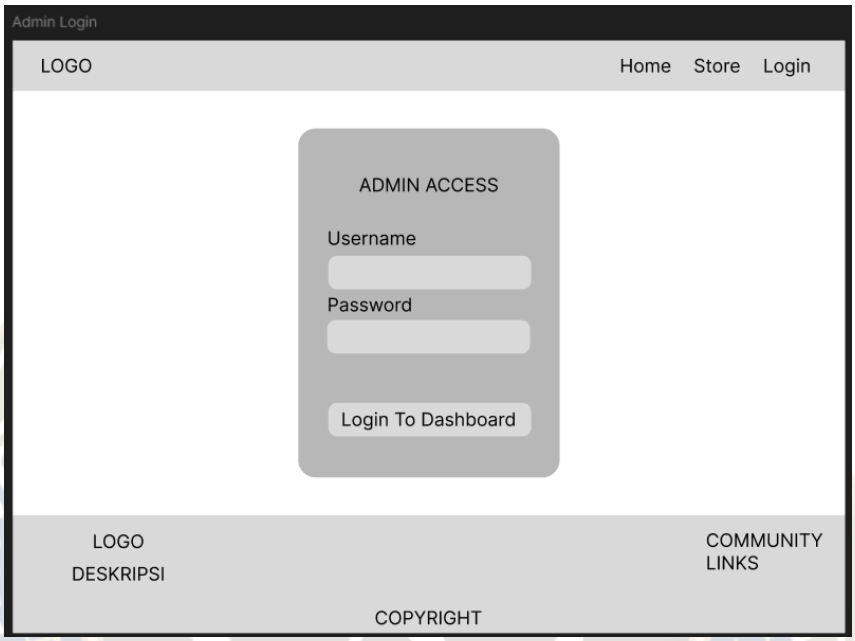


Gambar 3.19: *Wireframe Riwayat Transaksi User Page (UC-07)*

3.4.2 Antarmuka Administrator (Admin Interface)

3.4.2.1 Wireframe Login Admin

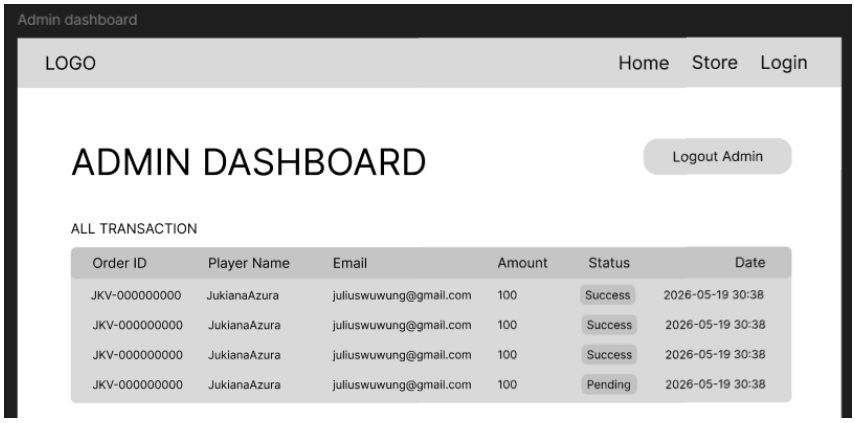
Halaman login khusus untuk administrator entri dengan validasi kredensial keamanan ganda yang berbeda dari gerbang pengguna biasa. Desain antarmukanya ditunjukkan pada Gambar 3.20.



Gambar 3.20: *Wireframe Login Admin*

3.4.2.2 Wireframe Riwayat Transaksi Global (UC-07)

Halaman panel khusus admin yang menyajikan tabel rekaman data seluruh *invoice* masuk, nominal omzet, beserta status mutasi riwayat pembayaran dari *callback gateway* Midtrans diilustrasikan pada Gambar 3.21.



Gambar 3.21: *Wireframe Riwayat Transaksi Admin Page (UC-07)*

3.4.2.3 Wireframe Provision Logs & Redeliver Tool (UC-08 & UC-09)

Antarmuka khusus admin yang menyajikan transparansi data eksekusi perintah API Pterodactyl ke server game ditunjukkan pada Gambar 3.22. Layar ini dilengkapi tombol aksi darurat khusus (*Redeliver*) untuk memicu pengiriman ulang paket item apabila status log mencatat kegagalan.



Gambar 3.22: *Wireframe Panel Provision Logs dan Fitur Redeliver (UC-08 & UC-09)*

BAB 4
IMPLEMENTASI

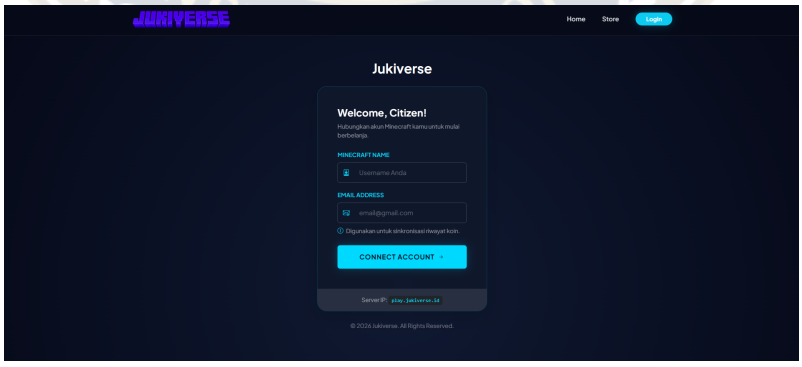
4.1 Implementasi Sistem

Bab ini membahas hasil implementasi sistem Jukiverse.store berdasarkan perancangan pada Bab 3. Implementasi dilakukan berdasarkan use case, perancangan basis data, sequence diagram, dan rancangan antarmuka yang telah dibuat sebelumnya. Pada tahap ini dijelaskan tampilan antarmuka sistem, alur penggunaan fitur, serta implementasi beberapa segmen program yang mendukung proses sistem.

Sistem dikembangkan menggunakan framework Laravel dengan bahasa pemrograman PHP untuk backend, serta HTML, CSS, dan JavaScript untuk frontend. Implementasi sistem mencakup fitur autentikasi pengguna, manajemen produk, proses pembelian koin, integrasi Midtrans sebagai payment gateway, serta integrasi Pterodactyl API untuk provisioning koin ke server Minecraft.

4.1.1 UC-01 & UC-02: Login via Minecraft Name dan Check Player Status

Fitur ini menangani proses autentikasi pengguna dengan memvalidasi nama Minecraft yang tersimpan pada database remote. Selain itu, sistem juga menerapkan mekanisme penguncian email agar satu akun Minecraft hanya dapat terhubung dengan satu alamat email. Tampilan awal dari halaman masuk ini ditunjukkan pada Gambar 4.1.



Gambar 4.1: *Antarmuka Halaman Utama Form Login Sistem Jukiverse.store*

Segmen Program

```
public function login(Request $request)
{
    $request->validate([
        'name' => 'required|string',
        'email' => 'required|email',
    ]);

    $name = trim($request->input('name'));
    $inputEmail = strtolower(trim($request->input('email')));

    $user = MinecraftUser::whereRaw(
        'LOWER(name) = ?',
        [strtolower($name)]
    )->first();

    if (!$user) {
        return back()->with(
            'error',
            'Username Minecraft tidak terdaftar!'
        );
    }

    $existingRecord = PlayerEmail::where(
        'minecraft_uuid',
        $user->uuid
    )->first();

    if ($existingRecord) {
        if (strtolower($existingRecord->email_address) !== $inputEmail) {
            return back()->with(
                'error',
                'Akun ini sudah terikat dengan email lain!'
            );
        }

        $existingRecord->update([
```

```
            'minecraft_name' => $user->name,
        ]);
    } else {
        PlayerEmail::create([
            'minecraft_uuid' => $user->uuid,
            'minecraft_name' => $user->name,
            'email_address' => $inputEmail,
        ]);
    }

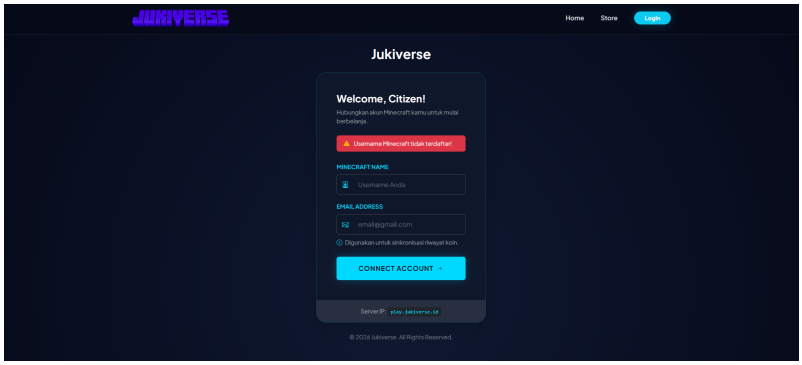
    session([
        'logged_in' => true,
        'uuid' => $user->uuid,
        'username' => $user->name,
        'email' => $inputEmail,
    ]);

    return redirect()->route('home');
}
```

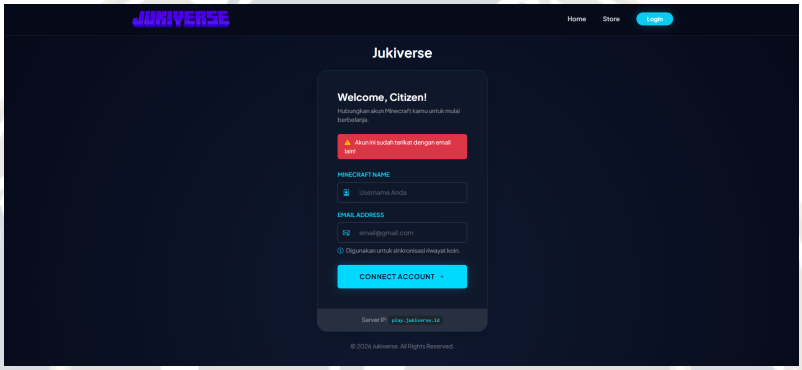
Segmen 4.1 *Implementasi Alur Login dan Verifikasi Player Remote*

Logika pemrograman pada Segmen 4.1 digunakan untuk memproses autentikasi pengguna berdasarkan nama Minecraft dan alamat email. Sistem terlebih dahulu melakukan validasi input, kemudian mencari data pemain pada database remote Minecraft. Apabila pemain tidak ditemukan di dalam basis data *remote*, sistem akan melemparkan pesan galat seperti pada Gambar 4.2.

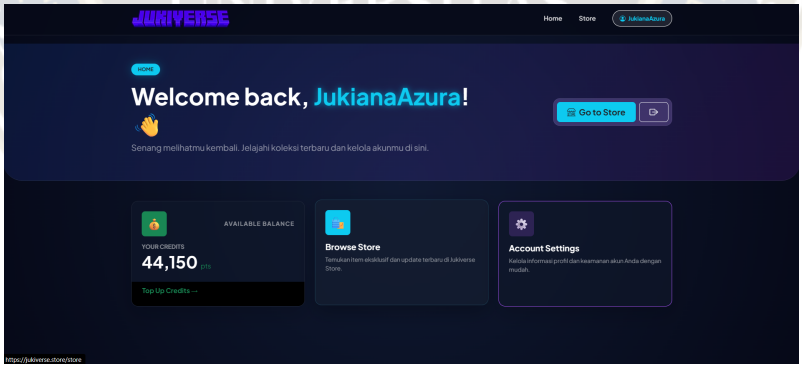
Selanjutnya, jika pengguna memasukkan email yang berbeda dari data yang sudah terkunci di dalam sistem, maka sistem akan menampilkan validasi eror seperti pada Gambar 4.3. Jika seluruh kredensial email dan nama akun sesuai, sistem akan membuat session login dan mengarahkan pengguna ke halaman utama dashboard seperti yang diilustrasikan pada Gambar 4.4.



Gambar 4.2: *Tampilan Validasi Eror Saat Username Minecraft Tidak Terdaftar*



Gambar 4.3: *Tampilan Validasi Eror Saat Akun Terikat dengan Alamat Email Lain*



Gambar 4.4: *Antarmuka Dashboard Pengguna Setelah Autentikasi Berhasil*

4.1.2 UC-03: Update Email

Fitur update email digunakan oleh pengguna untuk memperbarui alamat email yang terhubung dengan akun Minecraft. Perubahan ini hanya dilakukan pada database lokal website dan tidak mengubah data pada database remote Minecraft. Form pengaturan akun untuk memicu fitur ini dapat dilihat pada Gambar 4.5.

Segmen Program

```
public function updateEmail(Request $request)
{
    $request->validate([
        'email' => 'required|email|max:255',
    ]);

    $updated = PlayerEmail::where(
        'minecraft_uuid',
        session('uuid')
    )->update([
        'email_address' => $request->email,
        'updated_at' => now(),
    ]);

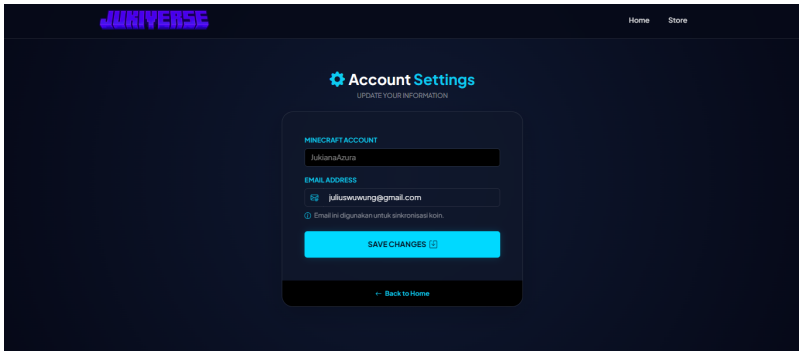
    if ($updated) {
        session(['email' => $request->email]);

        return back()->with(
            'success',
            'Email berhasil diperbarui!'
        );
    }

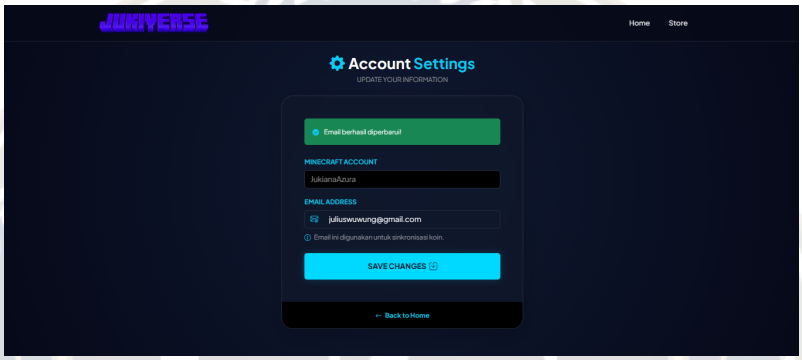
    return back()->with(
        'error',
        'Tidak ada perubahan atau data tidak ditemukan.'
    );
}
```

Segmen 4.2 *Implementasi Pembaruan Data Profil Pengguna*

Alur penulisan kode pada Segmen 4.2 digunakan untuk memperbarui email pengguna berdasarkan UUID Minecraft yang tersimpan pada session. Sistem melakukan validasi format email terlebih dahulu, kemudian memperbarui data pada tabel email. Jika proses berhasil dieksekusi, sistem menampilkan notifikasi sukses *flash message* seperti yang tampak pada Gambar 4.6.



Gambar 4.5: *Antarmuka Halaman Pengaturan Akun*



Gambar 4.6: *Tampilan Validasi dan Notifikasi Sukses Saat Mengganti Alamat Email*

4.1.3 UC-04: View Products

Fitur ini digunakan untuk menampilkan katalog produk aktif pada halaman store. Sistem hanya mengambil produk dengan status aktif dan mengecualikan produk khusus dengan ID 99 yang digunakan untuk pembelian custom coin.

Segmen Program

```
public function index()
{
    $products = Product::where('status', 'active')
        ->where('id', '!=', 99)
        ->get();

    $userEmail = PlayerEmail::where(
        'minecraft_uuid',
        session('uuid')
    )->first();
```

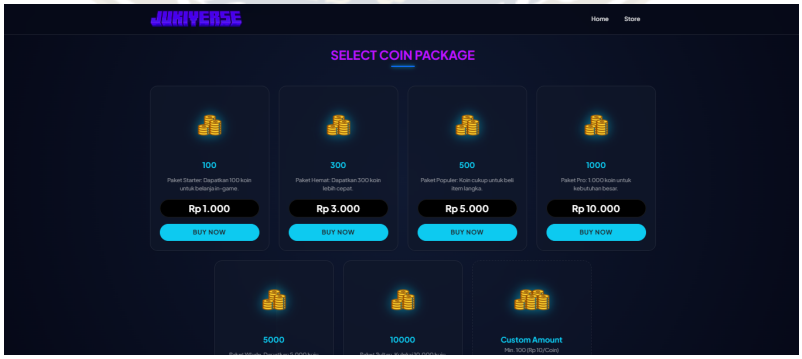
```
    $recentPurchases = [];

    if ($userEmail) {
        $recentPurchases = Transaction::where(
            'email_id',
            $userEmail->id
        )
            ->orderBy('created_at', 'desc')
            ->take(5)
            ->get();
    }

    return view('pages.store', [
        'products' => $products,
        'recentPurchases' => $recentPurchases,
    ]);
}
```

Segmen 4.3 *Implementasi Penampilan Katalog Produk*

Alur query data pada Segmen 4.3 digunakan untuk mengambil data produk aktif dari database dan menampilkannya pada halaman store. Selain itu, sistem juga mengambil data riwayat pembelian terbaru pengguna berdasarkan email yang terhubung dengan akun Minecraft. Hasil akhir implementasi visual dari katalog produk ini dipaparkan pada Gambar 4.7.



Gambar 4.7: *Antarmuka Hasil Implementasi Katalog Produk Jukiverse.store*

4.1.4 UC-05: Purchase Product

Fitur purchase product digunakan untuk memproses pilihan produk yang akan dibeli pengguna. Sistem mendukung pembelian produk fixed coin dan custom coin. Pada tahap ini sistem menyiapkan data checkout sebelum pengguna diarahkan ke proses pembayaran.

Segmen Program

```
public function store(Request $request)
{
    $type = $request->input('type');

    if ($type === 'fixed') {
        $product = Product::find($request->product_id);

        if (!$product) {
            return back()->with(
                'error',
                'Produk tidak ditemukan.'
            );
        }

        $coinAmount = (int) $product->product_name;

        $data = [
            'type' => 'fixed',
            'productId' => $product->id,
            'productName' => $coinAmount . " K-Bucks",
            'amount' => $coinAmount,
            'price' => $product->price,
        ];
    } else {
        $request->validate([
            'amount' => 'required|integer|min:100',
        ]);

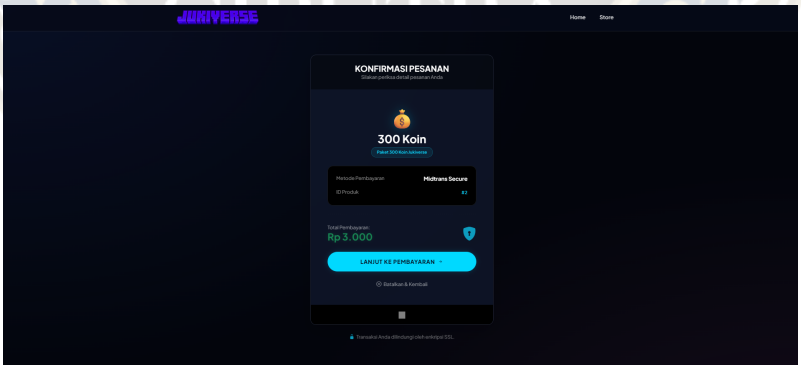
        $coins = $request->input('amount');
```

```
            $data = [
                'type' => 'custom',
                'productId' => null,
                'productName' => number_format($coins)
                    . " K-Bucks (Custom)",
                'amount' => $coins,
                'price' => $coins * 10,
            ];
        }

        return view('pages.checkout', $data);
    }
```

Segmen 4.4 *Implementasi Validasi Produk dan Halaman Checkout*

Fungsi kendali pada Segmen 4.4 digunakan untuk memproses data pembelian sebelum masuk ke halaman checkout. Jika pengguna memilih produk fixed coin, sistem akan mengambil data produk berdasarkan ID. Jika pengguna memilih custom coin, sistem akan melakukan validasi jumlah coin minimal dan menghitung harga berdasarkan jumlah coin yang dimasukkan. Hasil dari proses ini adalah penampilan rangkuman transaksi pada halaman konfirmasi pesanan seperti pada Gambar 4.8.



Gambar 4.8: *Antarmuka Halaman Konfirmasi Detail Pesanan Produk Toko Web*

Segmen Program

```
public function pay(Request $request)
{
    Config::$serverKey = env('MIDTRANS_SERVER_KEY');
```

```
Config::$isProduction = false;
Config::$isSanitized = true;
Config::$is3ds = true;

$orderId = 'JKV-' . time() . rand(10, 99);
$uuid = session('uuid');

$userEmail = PlayerEmail::where(
    'minecraft_uuid',
    $uuid
)->first();

if (!$userEmail) {
    return redirect()->route('store')->with(
        'error',
        'Data email player tidak ditemukan.'
    );
}

$productId = $request->product_id ?: 99;

Transaction::create([
    'email_id' => $userEmail->id,
    'product_id' => $productId,
    'minecraft_uuid' => $uuid,
    'minecraft_name' => session('username'),
    'midtrans_order_id' => $orderId,
    'payment_status' => 'pending',
    'amount' => $request->price,
]);

$params = [
    'transaction_details' => [
        'order_id' => $orderId,
        'gross_amount' => (int) $request->price,
    ],
```

```
            'customer_details' => [
                'first_name' => session('username'),
                'email' => session('email'),
            ],
        ];

        $snapToken = Snap::getSnapToken($params);

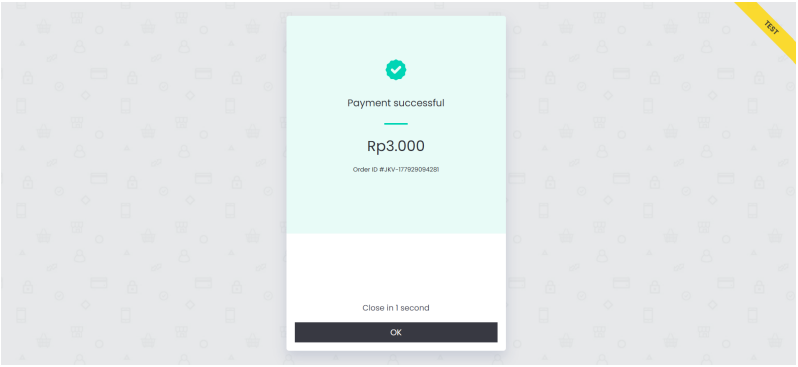
        return redirect()->away(
            "https://app.sandbox.midtrans.com/snap/v2/vtweb/"
            . $snapToken
        );
    }
```

Segmen 4.5 *Implementasi Pembuatan Transaksi Pembayaran Midtrans*

Kode program pada Segmen 4.5 bertanggung jawab penuh untuk menginisialisasi transaksi awal ke repositori lokal dan memicu REST API Midtrans. Setelah token Snap didapatkan, pengguna dialihkan langsung ke subsistem pembayaran Midtrans. Gambar 4.9 merepresentasikan tampilan *pop-up* integrasi pembayaran menggunakan metode otomatis QRIS GoPay. Apabila proses bayar selesai diselesaikan oleh klien, sistem menampilkan antarmuka sukses seperti pada Gambar 4.10.



Gambar 4.9: *Antarmuka Pop-up Snap Midtrans dengan Metode Pembayaran GoPay QRIS*



Gambar 4.10: *Tampilan Notifikasi Sistem Saat Transaksi Pembayaran Dinyatakan Berhasil*

4.1.5 UC-06: Update Credits

Fitur update credits merupakan proses otomatisasi pemberian koin kepada pemain setelah pembayaran dinyatakan berhasil. Sistem menerima callback dari Midtrans, memperbarui status transaksi, kemudian mengirim command ke server Minecraft melalui API Pterodactyl.

Segmen Program

```
public function callback(Request $request)
{
    $status = $request->transaction_status;
    $orderId = $request->order_id;

    if ($status == 'capture' || $status == 'settlement') {
        $transaction = Transaction::where(
            'midtrans_order_id',
            $orderId
        )->first();

        if ($transaction
            && $transaction->payment_status == 'pending') {

            $transaction->update([
                'payment_status' => 'success',
            ]);
```

```
            $coinAmount = (int) ($transaction->amount / 10);

            $this->sendToPterodactyl(
                $transaction->minecraft_uuid,
                $coinAmount,
                $orderId
            );
        }
    }

    return response()->json([
        'message' => 'Webhook received'
    ]);
}
```

Segmen 4.6 *Implementasi Sinkronisasi Callback Midtrans*

Fungsi *callback* pada Segmen 4.6 bertindak sebagai jembatan asinkron penangkap notifikasi instan dari Midtrans. Begitu data divalidasi, status lokal langsung diubah menjadi *success* dan otomatis mengeksekusi sub-fungsi pengiriman perintah ke konsol server target.

Segmen Program

```
private function sendToPterodactyl($uuid, $amount, $orderId)
{
    $transaction = Transaction::where(
        'midtrans_order_id',
        $orderId
    )->first();

    if (!$transaction || empty($transaction->minecraft_name)) {
        return;
    }

    $name = $transaction->minecraft_name;
    $command = "nextcredit give $name $amount";

    $response = Http::withHeaders([
```

```
            'Authorization' => 'Bearer ' . env('PTERO_API_KEY'),
            'Content-Type' => 'application/json',
            'Accept' => 'application/json',
        ])->post(
            env('PTERO_PANEL_URL')
            . "/api/client/servers/"
            . env('PTERO_SERVER_UUID')
            . "/command",
            [
                'command' => $command,
            ]
        );

        ProvisionLog::create([
            'transaction_id' => $transaction->id,
            'execution_status' => $response->successful()
                ? 'success'
                : 'failed',
            'executed_at' => now(),
            'message_log' => $response->successful()
                ? "Command executed: $command"
                : "Ptero Error: " . $response->body(),
        ]);
    }
```

Segmen 4.7 *Implementasi Otomatisasi Provisioning Koin*

Fungsi internal pada Segmen 4.7 bertugas melakukan *request* ke REST API panel Pterodactyl dengan membawa muatan command konsol game. Bukti keberhasilan otomatisasi penambahan saldo koin secara *real-time* langsung pada akun *in-game* pemain ditunjukkan pada Gambar 4.11.



Gambar 4.11: *Dokumentasi Hasil Otomatisasi Penambahan Saldo Koin pada Server Game*

4.1.6 UC-07: View Purchase History

Fitur riwayat transaksi dibagi menjadi dua bagian, yaitu tampilan pengguna dan tampilan administrator. Pengguna dapat melihat riwayat transaksi miliknya sendiri, sedangkan administrator dapat memantau seluruh transaksi yang masuk ke dalam sistem.

4.1.6.1 User View

Segmen Program

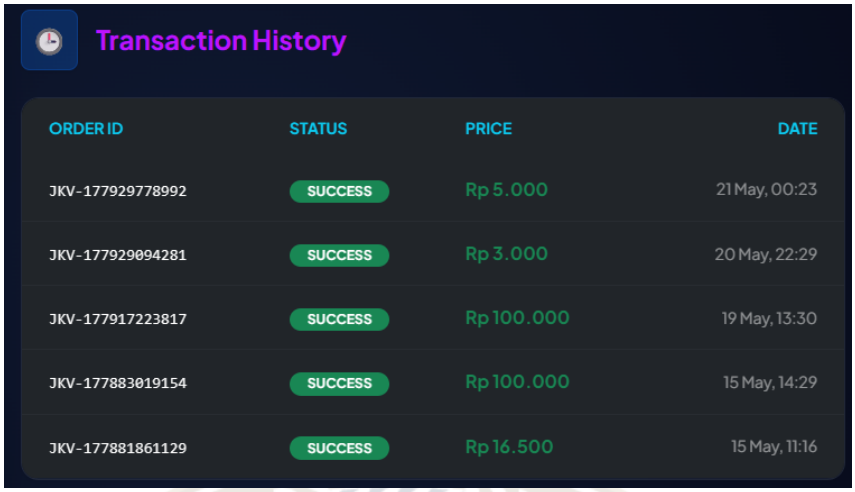
```
$userEmail = PlayerEmail::where(
    'minecraft_uuid',
    session('uuid')
)->first();

$recentPurchases = [];

if ($userEmail) {
    $recentPurchases = Transaction::where(
        'email_id',
        $userEmail->id
    )
        ->orderBy('created_at', 'desc')
        ->take(5)
        ->get();
}
```

Segmen 4.8 *Implementasi Riwayat Transaksi Pengguna*

Segmen program pada Segmen 4.8 digunakan untuk menarik data kepemilikan riwayat invoice pribadi pengguna. Hasil cetakan data riwayat ini direndering ke dalam antarmuka web khusus *User View* seperti yang ditunjukkan pada Gambar 4.12.



Gambar 4.12: *Antarmuka Riwayat Transaksi - User View*

4.1.6.2 Admin View

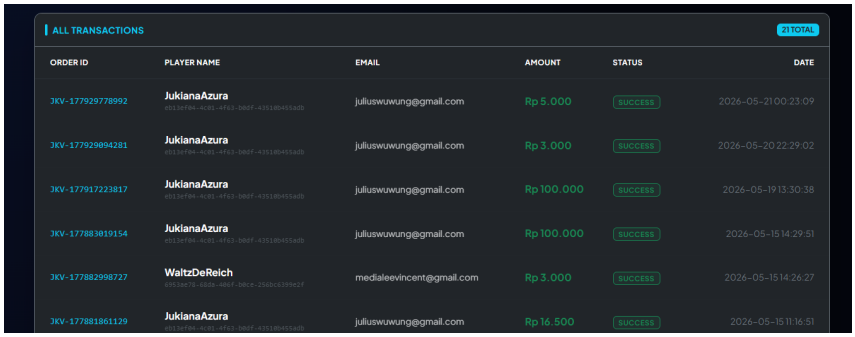
Segmen Program

```
$transactionsQuery = Transaction::query()
    ->leftJoin(
        'emails',
        'transactions.email_id',
        '=',
        'emails.id'
    )
    ->select('transactions.*', 'emails.email_address')
    ->orderBy('transactions.created_at', 'desc');

$transactions = $transactionsQuery->get();
```

Segmen 4.9 *Implementasi Log Transaksi Keseluruhan Administrator*

Proses penggabungan tabel (*join query*) pada Segmen 4.9 memfasilitasi kebutuhan pengelola dalam meninjau seluruh transaksi finansial secara menyeluruh. Panel log transaksi global versi *Admin View* diilustrasikan pada Gambar 4.13.



Gambar 4.13: *Antarmuka Log Transaksi - Admin View*

4.1.7 UC-08: View Provision Logs

Fitur provision logs digunakan oleh administrator untuk memantau hasil eksekusi command provisioning koin. Melalui fitur ini, administrator dapat melihat apakah command berhasil dikirim ke server Minecraft atau mengalami kegagalan.

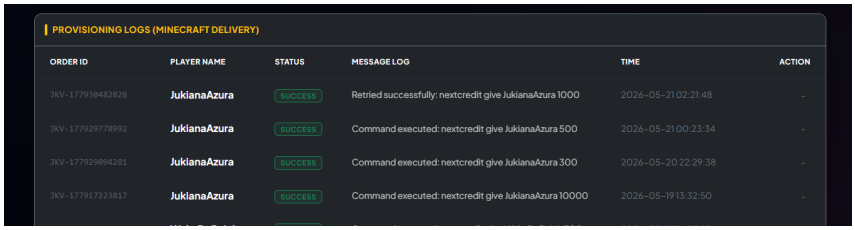
Segmen Program

```
$provisionLogsQuery = ProvisionLog::query()
    ->join(
        'transactions',
        'provision_logs.transaction_id',
        '=',
        'transactions.id'
    )
    ->select(
        'provision_logs.*',
        'transactions.midtrans_order_id',
        'transactions.minecraft_name',
        'transactions.minecraft_uuid'
    )
    ->orderBy('provision_logs.created_at', 'desc');

$provisionLogs = $provisionLogsQuery->get();
```

Segmen 4.10 *Implementasi Provision Logs Admin*

Segmen program pada Segmen 4.10 digunakan untuk memanggil data riwayat komunikasi API antara web store dengan Pterodactyl Wings Daemon. Implementasi antarmuka tabel monitoring log sistem ini dapat dilihat secara jelas pada Gambar 4.14.



Gambar 4.14: *Antarmuka Panel Monitoring Log Provisioning Administrator*

4.1.8 UC-09: Redeliver Failed Items

Fitur redeliver failed items digunakan untuk melakukan retry provisioning apabila proses pemberian koin sebelumnya gagal. Administrator dapat menekan tombol retry untuk mengirim ulang command ke server Minecraft melalui API Pterodactyl.

Segmen Program

```
public function retryProvision($id)
{
    $log = ProvisionLog::query()
        ->join(
            'transactions',
            'provision_logs.transaction_id',
            '=',
            'transactions.id'
        )
        ->select(
            'provision_logs.*',
            'transactions.amount',
            'transactions.minecraft_name',
            'transactions.midtrans_order_id'
        )
        ->where('provision_logs.id', $id)
        ->first();

    if (!$log) {
        return back()->with(
            'error',
            'Log tidak ditemukan.'
        );
```

```
}

$coinAmount = (int) ($log->amount / 10);
$command = "nextcredit give "
    . $log->minecraft_name
    . " "
    . $coinAmount;

$response = Http::withHeaders([
    'Authorization' => 'Bearer ' . env('PTERO_API_KEY'),
    'Content-Type' => 'application/json',
    'Accept' => 'application/json',
])->post(
    env('PTERO_PANEL_URL')
    . "/api/client/servers/"
    . env('PTERO_SERVER_UUID')
    . "/command",
    [
        'command' => $command,
    ]
);

if ($response->successful()) {
    ProvisionLog::where('id', $id)->update([
        'execution_status' => 'success',
        'message_log' => "Retried successfully: " . $command,
        'executed_at' => now(),
        'updated_at' => now(),
    ]);

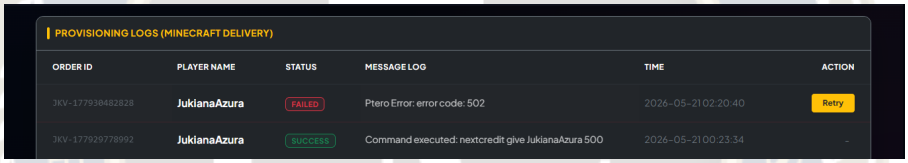
    return back()->with(
        'success',
        'Koin berhasil dikirim ulang.'
    );
}
```

```
        return back()->with(
            'error',
            'Gagal mengirim koin.'
        );
    }
```

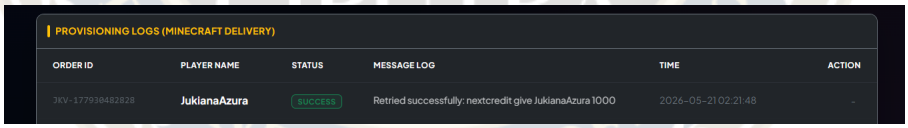
Segmen 4.11 *Implementasi Intervensi Manual Administrator*

Fungsi eksekusi ulang pada Segmen 4.11 digunakan untuk menangani kegagalan sistem pengiriman otomatis akibat gangguan jaringan eksternal. Gambar 4.15 mengilustrasikan tampilan log sistem ketika mencatat status gagal beserta kemunculan tombol aksi *Retry*.

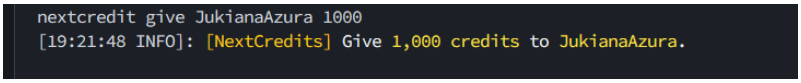
Apabila tombol tersebut ditekan dan API berhasil merespons dengan baik, status visual sistem akan berubah seketika menjadi sukses seperti pada Gambar 4.16. Keberhasilan intervensi manual ini juga dibuktikan dengan catatan log eksekusi pada sisi konsol server Pterodactyl yang ditunjukkan pada Gambar 4.17.



Gambar 4.15: *Antarmuka Log Transaksi Gagal dengan Opsi Tombol Retry*



Gambar 4.16: *Pembaruan Status Log Menjadi Success After Dilakukan Intervensi Manual*



Gambar 4.17: *Output Konsol Pterodactyl Menunjukkan Eksekusi Command dari Intervensi Manual Berhasil*

BAB 5
PENGUJIAN DAN PEMBAHASAN

5.1 Alpha Testing

Pengujian alpha (*Alpha Testing*) merupakan tahap pengujian internal yang dilakukan untuk memastikan seluruh fungsi pada sistem web store Minecraft berbasis Laravel berjalan sesuai dengan kebutuhan dan rancangan sistem yang telah dibuat. Pengujian ini bertujuan untuk menemukan kesalahan logika, bug, maupun ketidaksesuaian proses sebelum sistem digunakan oleh pengguna akhir.

Pengujian dilakukan terhadap fitur utama seperti autentikasi administrator, pengelolaan produk pada halaman store, proses pembelian koin, validasi pembelian custom coin, sinkronisasi status pembayaran melalui callback Midtrans, provisioning koin ke server Minecraft melalui API Pterodactyl, retry provisioning, riwayat transaksi pengguna, serta monitoring transaksi oleh administrator.

Dalam penelitian ini, pengujian alpha dibagi menjadi dua tahapan utama, yaitu *unit testing* dan *integration testing*. Unit testing dilakukan pada fungsi-fungsi utama yang dapat diuji secara terpisah, sedangkan integration testing dilakukan untuk memastikan komunikasi antar modul dan layanan eksternal dapat berjalan dengan baik. Pengujian dilakukan menggunakan PHPUnit, Composer, Visual Studio Code, Laravel Testing Framework, Midtrans Sandbox, dan API Pterodactyl.

5.1.1 Unit Testing

Unit testing dilakukan terhadap beberapa modul utama pada sistem Jukiverse, yaitu modul Authentication, Store, Purchase, Payment, Provision, Retry Provision, Transaction History, dan Admin Monitoring. Tujuan utama dari pengujian ini adalah untuk memastikan bahwa fungsi internal sistem berjalan sesuai dengan kebutuhan fungsional dan logika bisnis yang telah dirancang.

Pengujian unit difokuskan pada proses yang dapat diuji secara terpisah tanpa menjalankan keseluruhan alur integrasi sistem, seperti validasi password administrator, pengambilan produk aktif, validasi pembelian custom coin, validasi harga produk, perubahan status transaksi, penyimpanan Midtrans Order ID, perhitungan jumlah koin, pembuatan provision log, pembaruan status retry provisioning, pengambilan riwayat transaksi, serta pengecekan data transaksi untuk monitoring administrator.

Secara keseluruhan, unit testing pada sistem Jukiverse mencakup dua belas (12) test case dan seluruh pengujian berhasil dijalankan tanpa ditemukan kegagalan proses maupun bug pada logika utama sistem. Ringkasan dari sebaran data dan hasil eksekusi unit testing tersebut dapat dilihat pada Tabel 5.1 berikut. Adapun dokumentasi hasil pengujian secara lengkap dapat dilihat pada lampiran.

Table 5.1: *Ringkasan Hasil Unit Testing Sistem Jukiverse*

Modul	**Jumlah Test Case**	**Lulus**	**Gagal**	**Persentase**
Authentication	2	2	0	100%
Store	1	1	0	100%
Purchase	2	2	0	100%
Payment	2	2	0	100%
Provision	2	2	0	100%
Retry Provision	1	1	0	100%
Transaction History	1	1	0	100%
Admin Monitoring	1	1	0	100%
Total	**12**	**12**	**0**	**100%**

Berdasarkan hasil eksekusi unit testing yang telah dipetakan pada Tabel 5.1, terdapat beberapa temuan utama selama proses pengujian berlangsung, antara lain:

1. Seluruh fungsi internal utama pada sistem berhasil berjalan sesuai dengan kebutuhan dan logika bisnis yang telah dirancang.
2. Validasi password administrator berhasil memastikan proses autentikasi admin berjalan sesuai data yang tersimpan.
3. Pengambilan data produk aktif berhasil memastikan bahwa hanya produk dengan status aktif yang diproses oleh sistem.
4. Validasi pembelian custom coin berhasil menolak input jumlah coin yang tidak valid.
5. Perubahan status transaksi, penyimpanan Midtrans Order ID, serta pembuatan provision log berhasil berjalan sesuai dengan data transaksi.
6. Perhitungan jumlah koin berdasarkan nominal transaksi berhasil menghasilkan nilai yang sesuai dengan aturan sistem.

Adapun beberapa catatan dan rekomendasi untuk pengembangan sistem ke depannya berdasarkan matriks evaluasi Tabel 5.1, antara lain:

1. Disarankan untuk menambahkan pengujian terhadap edge cases seperti produk tidak ditemukan, email pengguna tidak tersedia, transaksi duplikat, serta nominal transaksi yang tidak valid.

2. Pengujian ulang perlu dilakukan apabila terdapat perubahan pada model, struktur database, logika transaksi, maupun proses provisioning.

3. Pengembangan selanjutnya dapat menambahkan automated frontend testing untuk memastikan kualitas tampilan dan interaksi pengguna.

5.1.2 Integration Testing

Integration testing dilakukan untuk memastikan bahwa proses komunikasi dan pertukaran data antar modul pada sistem Jukiverse dapat berjalan dengan baik secara terintegrasi. Pengujian dilakukan terhadap integrasi antara aplikasi web Laravel, database MySQL, Midtrans Payment Gateway, dan API Pterodactyl. Tujuan utama dari pengujian ini adalah memastikan bahwa seluruh proses bisnis sistem dapat berjalan secara sinkron tanpa mengalami kegagalan proses maupun kehilangan data selama sistem digunakan.

Integration testing dilakukan pada beberapa alur utama sistem seperti proses login administrator, pengambilan produk aktif pada halaman store, checkout produk fixed coin, sinkronisasi callback Midtrans terhadap status pembayaran, eksekusi command provisioning koin ke server Minecraft, retry provisioning pada transaksi gagal, pengambilan riwayat transaksi pengguna, monitoring transaksi dan provision log oleh administrator, serta validasi pembelian custom coin.

Secara keseluruhan, integration testing pada sistem Jukiverse mencakup sembilan (9) test case komparatif dan seluruh skenario pengujian berhasil dijalankan tanpa ditemukan error maupun kegagalan konektivitas data antarmodul. Rincian deskripsi materi alur data dan indikator kompleksitas skenario integration testing tersebut dipaparkan secara runtut pada Tabel 5.2.

Table 5.2: *Ringkasan Hasil Integration Testing Sistem Jukiverse*

No	Skenario Pengujian	Jumlah Modul Terlibat	Tingkat Kompleksitas
1	Proses login administrator dan pembuatan sesi login	3	Sedang – Melibatkan Authentication Module, Session Management, dan Database MySQL.
2	Pengambilan produk aktif pada halaman store	3	Sedang – Melibatkan Store Module, Product Model, dan Database MySQL.
3	Proses checkout produk fixed coin pada halaman purchase	4	Tinggi – Melibatkan Purchase Module, Product Model, Checkout View, dan Database MySQL.
4	Sinkronisasi callback Midtrans terhadap status pembayaran	4	Tinggi – Melibatkan Midtrans Callback Module, Transaction Model, Provision Module, dan Database MySQL secara asynchronous.
5	Eksekusi command provisioning koin ke server Minecraft	4	Tinggi – Melibatkan Provision Module, Transaction Model, Provision Log Model, dan API Pterodactyl.
6	Retry provisioning pada transaksi yang sebelumnya gagal	4	Tinggi – Melibatkan Admin Controller, Provision Log Module, API Pterodactyl, dan Database MySQL.
7	Pengambilan riwayat transaksi pengguna pada halaman store	3	Sedang – Melibatkan Transaction History Module, Store Module, dan Database MySQL.
8	Monitoring transaksi dan provision log oleh administrator	4	Sedang – Melibatkan Admin Dashboard Module, Transaction Model, Provision Log Model, dan Database MySQL.
9	Validasi pembelian custom coin pada sistem transaksi	3	Sedang – Melibatkan Purchase Module, Validation Module, dan Database MySQL.

Berdasarkan hasil peninjauan skenario pada Tabel 5.2, terdapat beberapa temuan utama selama proses pengujian hubungan antarmodul berlangsung, antara lain:

1. Seluruh proses komunikasi antara Laravel, database MySQL, Midtrans Payment Gateway, dan API Pterodactyl berhasil berjalan dengan baik tanpa dite-

mukan kehilangan data.

2. Integrasi Midtrans Payment Gateway berhasil melakukan sinkronisasi status pembayaran secara otomatis melalui callback notification.

3. Sistem provisioning berhasil mengirimkan command pemberian koin ke server Minecraft melalui API Pterodactyl sesuai dengan status pembayaran transaksi.

4. Proses retry provisioning berhasil memperbarui status provision log dari *failed* menjadi *success* ketika proses pengiriman ulang berhasil dilakukan.

5. Riwayat transaksi pengguna dan dashboard monitoring administrator berhasil menampilkan data transaksi serta provision log sesuai dengan data pada database.

6. Validasi pembelian custom coin berhasil menolak input yang tidak sesuai dengan aturan minimum pembelian.

Adapun beberapa rekomendasi pengembangan lanjutan pasca-analisis data integrasi pada Tabel 5.2 antara lain:

1. Integration testing perlu dilakukan kembali apabila terdapat perubahan pada struktur database, route, controller, model, maupun layanan eksternal yang digunakan sistem.

2. Pengembangan selanjutnya dapat menambahkan stress testing and load testing untuk mengukur kemampuan sistem dalam menangani jumlah transaksi yang lebih besar.

3. Monitoring terhadap layanan eksternal seperti Midtrans dan API Pterodactyl perlu dilakukan secara berkala untuk menjaga kestabilan sistem integrasi.

4. Pengembangan berikutnya dapat menambahkan automated frontend testing untuk meningkatkan kualitas sistem secara keseluruhan.

5.2 Beta Testing

Pengujian beta (*Beta Testing*) merupakan tahap pengujian yang dilakukan secara langsung kepada calon pengguna untuk mengetahui kesesuaian sistem dengan kebutuhan operasional pengguna akhir. Pengujian ini bertujuan untuk memperoleh umpan balik terkait fungsionalitas, kemudahan penggunaan, efektivitas fitur, serta

pengalaman pengguna dalam menggunakan sistem web store Minecraft berbasis Laravel.

Berbeda dengan pengujian alpha yang berfokus pada aspek teknis internal sistem, pengujian beta lebih menitikberatkan pada pengalaman penggunaan sistem dalam kondisi operasional nyata. Pada penelitian ini, pengujian beta dilakukan menggunakan pendekatan *closed beta testing* dengan melibatkan administrator server dan pemain aktif sebagai responden pengujian.

Pengujian dilakukan terhadap fitur utama sistem seperti login menggunakan username Minecraft dan email, akses halaman store, pembelian koin, proses pembayaran melalui Midtrans Sandbox, penerimaan koin di server Minecraft, serta pengelolaan transaksi melalui dashboard administrator. Tujuan pengujian ini adalah untuk mengetahui apakah sistem yang dikembangkan mampu membantu proses pembelian koin secara otomatis, terstruktur, dan efisien tanpa intervensi manual administrator.

Selain itu, standar kualitas perangkat lunak ISO/IEC 25010 digunakan sebagai acuan dalam penyusunan instrumen pengujian beta untuk mengevaluasi kualitas sistem dari sisi *functional suitability*, *performance efficiency*, *usability*, *reliability*, *effectiveness*, dan *satisfaction*.

5.2.1 Skema Pengujian

Pengujian beta pada sistem web store Minecraft dilakukan menggunakan metode *closed beta testing* untuk mengetahui tingkat kesesuaian sistem terhadap kebutuhan pengguna akhir dalam melakukan pembelian koin secara online. Pengujian dilakukan dengan melibatkan perwakilan penilai dengan komposisi subjek penguji yang dirangkum pada Tabel 5.3.

Table 5.3: *Profil Penguji Beta Testing*

No	**Profil/Peran**	**Jumlah**
1	Administrator Server	1 Orang
2	Pemain Aktif	4 Orang

Berdasarkan pembagian peran pada Tabel 5.3, para penguji diminta mencoba fitur utama sistem seperti login menggunakan username Minecraft dan email, melihat produk pada halaman store, melakukan pembelian koin, menyelesaikan pembayaran menggunakan Midtrans Sandbox, memastikan koin diterima di dalam server Minecraft, serta memeriksa status transaksi melalui dashboard administrator. Sete-

lah proses pengujian selesai, dilakukan sesi wawancara singkat untuk memperoleh masukan kualitatif yang mendalam.

5.2.2 Instrumen Wawancara Ujicoba

Instrumen wawancara pada pengujian beta disusun berdasarkan standar kualitas perangkat lunak ISO/IEC 25010. Instrumen ini digunakan untuk memperoleh masukan dari pengguna terkait kualitas sistem, kemudahan penggunaan, efektivitas fitur, serta pengalaman pengguna selama menggunakan sistem web store Minecraft.

Beberapa aspek kualitas produk yang dievaluasi dirumuskan ke dalam draf pertanyaan wawancara khusus yang ditunjukkan pada Tabel 5.4.

Table 5.4: *Pertanyaan Wawancara untuk Model Kualitas Produk*

Kriteria ISO/IEC 25010	**Pertanyaan**	**Tujuan**
Functional Suitability	Apakah fitur utama seperti login, store, pembayaran, dan penerimaan koin sudah berjalan sesuai kebutuhan?	Mengetahui kesesuaian fungsi sistem dengan kebutuhan pengguna.
Functional Suitability	Apakah terdapat fitur yang tidak berjalan atau mengalami kendala saat digunakan?	Mengidentifikasi kesalahan fungsi atau bug pada sistem.
Performance Efficiency	Apakah proses pembayaran hingga koin diterima di server berjalan dengan cepat?	Mengetahui performa sistem dalam memproses transaksi.
Usability	Apakah terdapat bagian tampilan atau alur sistem yang membingungkan?	Mengidentifikasi kendala penggunaan antarmuka.
Reliability	Apakah sistem pernah mengalami error, gagal transaksi, atau koin tidak masuk setelah pembayaran berhasil?	Mengetahui kestabilan sistem saat digunakan.

Selanjutnya, untuk mengukur parameter efektivitas serta tingkat kepuasan akhir dari pengguna ekosistem aplikasi, penyusunan pertanyaan wawancara model kualitas penggunaan dijabarkan pada Tabel 5.5.

Table 5.5: *Pertanyaan Wawancara untuk Model Kualitas Penggunaan*

Kriteria ISO/IEC 25010	**Pertanyaan**	**Tujuan**
Effectiveness	Apakah sistem membantu proses pembelian koin menjadi lebih mudah dan terstruktur?	Mengetahui efektivitas sistem dalam mendukung transaksi pemain.
Effectiveness	Apakah penggunaan sistem mengurangi kebutuhan bantuan manual dari administrator?	Mengukur peningkatan efektivitas operasional server.
Satisfaction	Apakah Anda merasa puas dengan fitur dan pengalaman penggunaan sistem web store Minecraft?	Mengetahui tingkat kepuasan pengguna terhadap sistem.
Satisfaction	Fitur apa yang menurut Anda paling membantu dalam sistem ini?	Mengidentifikasi fitur yang paling bermanfaat bagi pengguna.

5.3 Pembahasan

Berdasarkan hasil uji coba beta (*beta testing*) yang dilakukan bersama sejumlah responden yang merepresentasikan peran pengguna (*player*) dan pengelola (*staff*), diperoleh data empiris yang valid mengenai performa dan utilitas sistem Jukiverse.store. Secara keseluruhan, seluruh responden berhasil menyelesaikan proses autentikasi, penjelajahan katalog, hingga simulasi penyelesaian transaksi pembayaran menggunakan QRIS secara mandiri tanpa membutuhkan intervensi atau bantuan teknis eksternal.

Hasil umpan balik dari fase pengujian tersebut kemudian dianalisis dan dikelompokkan ke dalam beberapa parameter evaluasi, meliputi aspek keandalan fungsional, analisis waktu tanggap pengiriman produk, tingkat efisiensi operasional pengelola, serta evaluasi antarmuka dan rekomendasi pengembangan sistem.

5.3.1 Analisis Keandalan Fungsional dan Kemudahan Pembayaran

Evaluasi kualitatif menunjukkan bahwa sistem telah mampu memenuhi kebutuhan fungsional inti dengan sangat baik. Seluruh alur utama pengoperasian dinilai ber-

jalan normal dan sesuai dengan kebutuhan ekosistem server game. Integrasi *payment gateway* menggunakan metode QRIS terbukti memberikan kemudahan transaksi yang signifikan, mengingat metode pembayaran tersebut sangat familiar dan sesuai dengan preferensi mayoritas target pengguna saat ini.

Selama periode pengujian intensif dijalankan, tidak ditemukan kendala teknis seperti kegagalan sistem (*system crash*) ataupun kemacetan data pada modul pemrosesan transaksi maupun pada enkapsulasi data halaman konfirmasi pesanan. Kendati demikian, terdapat catatan kritis mengenai perlunya penguatan aspek keamanan (*security hardening*) pada fase produksi lanjutan untuk memitigasi potensi celah eksploitasi pada gerbang autentikasi nama pemain.

5.3.2 Analisis Waktu Tanggap Pengiriman Koin (Latency)

Parameter keberhasilan utama dari otomatisasi platform ini terletak pada minimalisasi waktu tunggu konfirmasi pengiriman produk setelah pembayaran dinyatakan sukses. Berdasarkan data rekam jejak selama pengujian, rata-rata waktu tanggap (*response time*) yang dibutuhkan dari momen suksesnya konfirmasi pembayaran di sisi *payment gateway* hingga perintah eksekusi API selesai dijalankan oleh *Wings Daemon* pada konsol server game adalah berkisar antara 1 hingga 5 detik.

Akselerasi performa pengiriman yang instan ini membuktikan efektivitas implementasi arsitektur *Webhook* asinkron pada framework backend yang digunakan. Mekanisme ini terbukti mampu memotong birokrasi verifikasi manual yang sebelumnya menjadi hambatan utama dalam efisiensi waktu operasional.

5.3.3 Analisis Efisiensi Operasional Pengelola dan Fitur Unggulan

Dari sudut pandang manajemen operasional, otomatisasi ini terbukti secara signifikan menurunkan beban kerja (*workload*) harian dari jajaran staff pengelola. Sistem ini berhasil mengeleminasi aktivitas pemantauan mutasi rekening secara manual yang rentan terhadap kelalaian manusia (*human error*). Terdapat dua modul utama yang dinilai memberikan dampak efisiensi tertinggi:

- **Modul Riwayat Transaksi (*Purchase History*):** Fitur ini memfasilitasi pengguna untuk melakukan pelacakan (*tracking*) mandiri terhadap status invoice dan item yang telah dibeli, sekaligus menjadi transparansi rekaman data yang valid bagi pihak internal pengelola.
- **Panel Log Pengiriman (*Provision Logs* dan *Redeliver Tool*):** Modul ini dinilai sebagai solusi taktis terbaik untuk mengantisipasi potensi isu eksternal,

seperti gangguan jaringan atau kondisi server game yang mendadak tidak aktif (*stuck*) di tengah jalan transaksi. Keberadaan fungsi *Retry Provision* pada panel administrator mempermudah staff melakukan intervensi manual guna mengirim ulang perintah koin yang sempat tertahan hanya dengan satu kali aksi.

5.3.4 Evaluasi Antarmuka (UI/UX) dan Rekomendasi Pengembangan

Meskipun arsitektur fungsi utama sistem dinilai berhasil, proses pengujian tetap menghasilkan beberapa poin evaluasi dan saran konstruktif terkait tata letak (*layout*), kontras visual, serta ekspansi fitur komersial untuk pengembangan berikutnya:

1. **Restrukturisasi Hierarki Halaman Toko:** Tata letak halaman toko perlu diatur ulang agar lebih intuitif bagi pengguna baru. Komponen katalog produk disarankan untuk dipindahkan ke area atas layar agar langsung terlihat sesaat setelah pengguna berhasil masuk ke dalam sistem, sedangkan komponen riwayat pembelian dapat diletakkan pada bagian bawah atau dialokasikan pada halaman tersendiri.

2. **Penyederhanaan Elemen Navigasi (*Clean UI*):** Beberapa tombol navigasi pasca-login dinilai redundan dan berpotensi membingungkan pengguna. Pembersihan elemen papan ucapan selamat datang (*welcome sign*) serta penempatan tombol keluar (*log out*) secara konvensional pada bar navigasi bagian atas (*navbar*) sangat disarankan demi estetika antarmuka yang lebih rapi.

3. **Kontras Warna dan Saluran Bantuan:** Kombinasi warna pada beberapa komponen visual dinilai terlalu kontras sehingga mengurangi kenyamanan visual dalam durasi akses yang lama. Selain itu, diperlukan penambahan informasi kontak person atau pusat bantuan (*helpdesk*) yang jelas di area bawah website sebagai jalur mitigasi pengaduan darurat apabila terjadi kendala transaksi.

4. **Diversifikasi Komoditas Toko:** Terkait perluasan potensi komersial, disarankan agar platform tidak membatasi sistem penjualan pada koin digital saja. Pengembangan ke depan diharapkan dapat mengintegrasikan fitur penjualan item non-koin secara langsung, serta otomatisasi paket pangkat keanggotaan (*rank*) khusus di dalam server game.

5.3.5 Temuan Bugs dan Tindak Lanjut

Selama pengujian operasional berlangsung, sistem mencatat beberapa hambatan berskala minor dari sisi kegagalan fungsi internal maupun kendala pengalaman pengguna (*usability*). Evaluasi komprehensif mengenai daftar anomali beserta rumusan solusi perbaikannya dipaparkan pada Tabel 5.6.

Table 5.6: *Temuan Bugs dan Tindak Lanjut*

No	**Permasalahan**	**Kategori**	**Solusi**
1	Terjadi kegagalan sistem pada fitur *custom product* yang menyebabkan nilai *product_id* menjadi *null* saat transaksi.	Bugs	Menetapkan ID 99 sebagai identitas produk khusus untuk menjamin integritas relasi database.
2	Penggunaan warna *palette* kurang cocok dengan tema Minecraft sehingga beberapa pengguna merasa kurang menarik.	Usability	Mengganti warna dengan palet yang lebih gelap dan kontras untuk meningkatkan estetika visual.
3	Footer pada halaman website terlalu kosong dan terlalu besar sehingga memberikan kesan tidak profesional.	Usability	Mengoptimalkan desain footer untuk memberikan tampilan yang lebih profesional.
4	Pada halaman *login* di bagian navbar terdapat tombol *login* yang tidak berfungsi sehingga membingungkan pengguna.	Bugs	Memperbaiki fungsi tombol *login* untuk memastikan operasional berjalan normal.

Analisis akhir yang bersandar pada kompilasi penyelesaian masalah pada Tabel 5.6 menunjukkan bahwa sistem telah memenuhi kebutuhan utama yaitu otomatisasi pembelian koin tanpa intervensi manual administrator selama 24/7.

BAB 6
KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil rancang bangun, implementasi, dan pengujian yang telah dilakukan pada sistem toko web Minecraft otomatis (Jukiverse), dapat ditarik kesimpulan sebagai berikut:

1. Sistem telah berhasil mengotomatisasi seluruh siklus pembelian layanan di server Minecraft, mulai dari verifikasi identitas pemain hingga pengiriman koin ke dalam game tanpa memerlukan intervensi manual dari administrator secara *real-time*.
2. Integrasi Midtrans API sebagai gerbang pembayaran memungkinkan sistem menerima berbagai metode pembayaran digital yang umum digunakan di Indonesia, yang secara langsung meningkatkan aksesibilitas bagi pemain.
3. Penggunaan Pterodactyl API terbukti efektif sebagai jembatan eksekusi perintah (*command execution*) dari platform web ke konsol server Minecraft dengan tingkat keberhasilan eksekusi yang tinggi dan *latency* yang rendah (di bawah 5 detik).
4. Mekanisme *lock-in email* pada sistem autentikasi berhasil menjaga integritas data dan mencegah terjadinya kesalahan pengiriman item akibat perbedaan identitas antara akun web dan akun dalam game.
5. Berdasarkan hasil *Beta Testing*, sistem dinyatakan layak dan memenuhi standar fungsionalitas (*Functional Suitability*) serta kemudahan penggunaan (*Usability*) sesuai dengan kebutuhan pemain dan pemilik server.

6.2 Saran

Meskipun sistem ini telah berfungsi dengan baik sesuai dengan lingkup penelitian, terdapat beberapa keterbatasan yang dapat dikembangkan lebih lanjut pada penelitian di masa mendatang:

1. **Integrasi Discord Bot:** Mengembangkan sistem notifikasi otomatis yang terhubung dengan bot Discord komunitas Jukiverse, guna memberikan pembaruan status transaksi sukses secara *real-time* kepada pemain serta administrator untuk meningkatkan transparansi layanan.

2. **Ekspansi Varian Produk Non-Koin:** Menambahkan fitur manajemen katalog produk agar sistem toko web tidak hanya melayani pengisian koin (*virtual currency*), melainkan juga mendukung penjualan item digital lain seperti *rank*, paket *permission*, maupun *crate keys* langsung ke dalam *game*.

3. **Dukungan Aksesibilitas Versi Bedrock Edition:** Mengembangkan kompatibilitas sistem pengiriman item (*delivery execution*) agar dapat mendukung pemain dari platform Minecraft Bedrock Edition (PE/Konsol), termasuk penanganan konversi karakter khusus pada *username* seperti tanda awalan (*prefix*) Geyser.

4. **Menambahan Fitur Redeem Code:** Mengimplementasikan sistem *redeem code* yang memungkinkan pemain mendapatkan kode unik untuk medapatkan potongan harga saat melakukan pembelian.

UNIVERSITAS CIPUTRA
ENTREPRENEURS & PROFESSIONALS

DAFTAR PUSTAKA

Everitt, D. (2016). *Getting started*. Pterodactyl. https://pterodactyl.io/panel/1.0/getting_started.html

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Hamdana, E. N., & Apriyani, M. E. (2020). Analisis Implementasi RESTfull Web Service Menggunakan Resource-Oriented Architecture. *Jurnal Informatika Polinema*, 6(2), 57–60. https://doi.org/10.33795/jip.v6i2.335

Lehdonvirta, V. (2009). Virtual item sales as a revenue model: Identifying attributes that drive purchase decisions. *Electronic Commerce Research*, 9(1), 97–113. https://doi.org/10.1007/s10660-009-9048-2

Nurhayati, M., & Setiawan, R. (2024). *Implementasi Payment Gateway Midtrans Pada Aplikasi Toko Mas Berbasis Web*. Jurnal Algoritma. https://jurnal.itg.ac.id/index.php/algoritma/article/view/1686

Perdana, B. A. (2024). *Rancang Bangun Sistem Informasi Top Up Game Online Berbasis Web dengan Framework Laravel (Studi Kasus Cheezy Store)*. Universitas Teknologi Digital Indonesia. https://eprints.utdi.ac.id/10623/

Pressman, R. S. (2015). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.

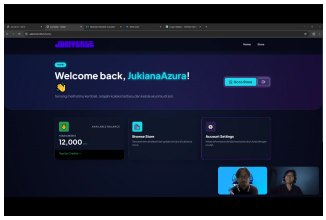
Ramadhani, D. A., Nugraha, P. E., Syahrizal, M. I., & Sani, R. R. (2025). *Perancangan Aplikasi E-Commerce Top-Up Game Online Berbasis Website dengan Integrasi Payment Gateway*. Jurnal Teknologi dan Informasi. https://ojs.unikom.ac.id/index.php/jati/article/view/17055

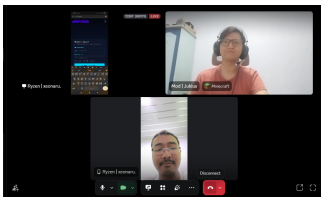
Sasmito, G. W. (2017). Penerapan Metode Waterfall Pada Desain Sistem Informasi Geografis Industri Kabupaten Tegal. *Jurnal Informatika: Jurnal Pengembangan IT*, 2(1), 6–12. https://doi.org/10.30591/jpit.v2i1.435

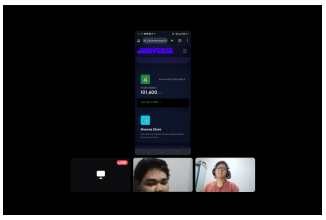
Stauffer, M. (2019). *Laravel: Up & Running: A Gentle Introduction to the World's Most Popular PHP Framework*. O'Reilly Media.

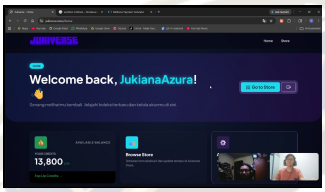
LAMPIRAN A

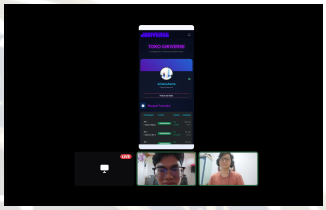
Dokumentasi Beta Testing











Lampiran ini berisi dokumentasi kegiatan beta testing dan pengumpulan data yang dilakukan dengan melibatkan 1 (satu) orang Administrator Server serta 4 (empat) orang pengguna yang berpartisipasi dalam tahap *beta testing*. Dokumentasi ini disertakan sebagai bukti pelaksanaan proses pengumpulan data secara langsung dan valid yang digunakan dalam penelitian. Seluruh kegiatan beta testing dan diskusi dilakukan secara daring melalui platform Discord tanpa tatap muka secara langsung, mengingat beberapa narasumber berada di luar Kota Surabaya sehingga tidak memungkinkan untuk dilakukan pertemuan secara langsung.

LAMPIRAN B

Dokumentasi Laporan Hasil Unit Testing

LAPORAN HASIL UNIT TESTING

Nama Proyek	: Jukiverse
Modul yang Diuji	:
	Authentication Store
	Purchase
	Payment
	Provision
	Retry Provision
	Transaction History
	Admin Monitoring
Tanggal Pengujian	: 31 Mei 2026
Penguji	: Jessen
Tools yang Digunakan	: PHPUnit, Composer, Visual Studio Code
Framework	: Laravel

1. Tujuan Pengujian

Pengujian unit dilakukan untuk memastikan bahwa fungsi-fungsi utama pada sistem Jukiverse berjalan sesuai dengan kebutuhan fungsional dan logika bisnis yang telah dirancang. Pengujian ini difokuskan pada logika internal sistem seperti autentikasi admin, validasi produk, validasi pembelian custom coin, perubahan status transaksi, perhitungan jumlah coin, pencatatan provision log, serta pengambilan riwayat transaksi.

Selain itu, pengujian ini bertujuan untuk memastikan bahwa data yang diproses oleh model dan database lokal dapat berjalan dengan benar tanpa harus menguji keseluruhan alur integrasi sistem seperti routing, controller penuh, Midtrans Callback, maupun Pterodactyl API.

2. Daftar Test Case

No	Nama Test Case	Fungsi yang Diuji	Deskripsi	Input	Output yang Diharapkan	Status
Modul Authentication						
1	testLoginAdminPasswordIsValid()	Admin Password Validation	Memvalidasi password administrator yang tersimpan dalam database.	Password valid	Password berhasil diverifikasi	Passed
2	testLoginAdminPasswordIsInvalid()	Admin Password Validation	Menolak password administrator yang tidak sesuai.	Password tidak valid	Password berhasil ditolak	Passed
Modul Store						
3	testStoreOnlyReturnsActiveProduct()	Product Model	Mengambil produk dengan status aktif.	Data produk aktif dan nonaktif	Hanya produk aktif yang ditampilkan	Passed
Modul Purchase						
4	testCustomCoinPurchaseValidation()	Purchase Validation	Memvalidasi jumlah coin pada pembelian custom coin.	Jumlah coin tidak valid	Input ditolak oleh sistem	Passed
5	testFixedProductPriceIsValid()	Product Validation	Memvalidasi harga produk fixed coin.	Data produk fixed coin	Harga produk sesuai data	Passed
Modul Payment						
6	testPaymentStatusCanBeUpdatedToSuccess()	Transaction Model	Mengubah status transaksi dari pending menjadi success.	Transaksi pending	Status transaksi berubah menjadi success	Passed
7	testMidtransOrderIdIsSavedCorrectly()	Transaction Model	Memastikan Midtrans Order ID tersimpan dengan benar.	Order ID transaksi	Order ID berhasil tersimpan	Passed
Modul Provision						
8	testProvisionCommandAmountCalculation()	Provision Calculation	Menghitung jumlah coin berdasarkan nominal transaksi.	Nominal transaksi Rp10.000	Jumlah coin = 1000	Passed
9	testProvisionLogCanBeCreated()	ProvisionLog Model	Membuat log provisioning setelah transaksi berhasil.	Data transaksi valid	Provision log berhasil dibuat	Passed
Modul Retry Provision						
10	testRetryProvisionStatusCanBeUpdated()	ProvisionLog Model	Mengubah status provisioning dari failed menjadi success.	Provision log failed	Status berhasil diperbarui	Passed
Modul Transaction History						
11	testTransactionHistoryCanBeFilteredByEmailId()	Transaction Model	Mengambil riwayat transaksi berdasarkan email pengguna.	Email ID pengguna	Riwayat transaksi berhasil ditemukan	Passed
Modul Admin Monitoring						
12	testAdminTransactionMonitoringDataExists()	Transaction Model	Memastikan data transaksi tersedia untuk monitoring admin.	Data transaksi	Data berhasil ditemukan	Passed

3. Rangkuman Hasil

Total Test Case	: 12
Assertions	: 14
Lulus / Passed	: 12
Gagal	: 0
Persentase Keberhasilan	: 100%

4. Catatan Penting

Seluruh fungsi utama yang diuji pada unit testing berhasil berjalan sesuai dengan kebutuhan sistem. Pengujian ini membuktikan bahwa logika dasar pada modul Authentication, Store, Purchase, Payment, Provision, Retry Provision, Transaction History, dan Admin Monitoring dapat berjalan dengan baik.

Pengujian unit ini difokuskan pada proses yang dapat diuji secara terpisah, seperti validasi password admin, pengambilan produk aktif, validasi pembelian custom coin, perubahan status transaksi, perhitungan jumlah coin, pembuatan provision log, serta pengambilan data transaksi. Sementara itu, proses yang melibatkan interaksi antar komponen seperti routing, controller, database, Midtrans Callback, dan Pterodactyl API diuji secara terpisah melalui integration testing.

5. Rekomendasi

Pengujian ulang perlu dilakukan apabila terdapat perubahan pada struktur database, model, validasi transaksi, maupun logika provisioning coin. Selain itu, pengujian dapat dikembangkan dengan menambahkan skenario edge case, seperti transaksi dengan nominal tidak valid, produk tidak aktif, data email tidak ditemukan, serta kegagalan proses provisioning.

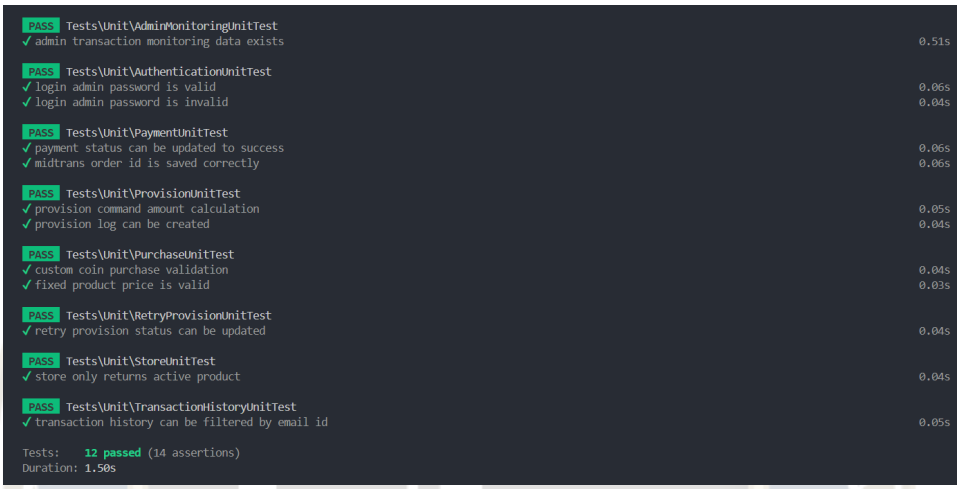
6. Contoh Kode Pengujian

```
public function test_admin_transaction_monitoring_data_exists(): void
{
    $transaction = $this->createTransaction('success', 'ORDER-ADMIN');

    $this->assertDatabaseHas('transactions', [
```

```
            'id' => $transaction->id,
            'minecraft_name' => 'Steve',
            'payment_status' => 'success',
        ]);
    }
```

Screenshot Log Pengujian



UNIVERSITAS CIPUTRA
ENTREPRENEURS & PROFESSIONALS

LAMPIRAN C

Dokumentasi Laporan Hasil Integration Testing

LAPORAN HASIL INTEGRATION TESTING

Nama Proyek	: Jukiverse Web Store
Modul yang Diuji	:
	Modul Authentication Integration
	Modul Product Purchase Integration
	Modul Midtrans Callback Integration
	Modul Provision System Integration
	Modul Pterodactyl API Integration
	Modul Transaction History Integration
Tanggal Pengujian	: 25 Mei 2026
Penguji	: Julius Sebastian Wuwung
Tools yang Digunakan	: PHPUnit, Composer, Visual Studio Code
Framework	: Laravel

1. Tujuan Pengujian

1. Memastikan bahwa proses komunikasi data antara website Laravel, database MySQL, Midtrans Payment Gateway, dan Pterodactyl API dapat berjalan dengan baik secara terintegrasi.
2. Memastikan bahwa proses autentikasi pengguna menggunakan Minecraft Name dapat berjalan sesuai kebutuhan sistem.
3. Memastikan bahwa integrasi sistem pembayaran menggunakan Midtrans dapat berjalan dengan baik pada proses pembuatan transaksi pembayaran.
4. Memastikan bahwa callback notification dari Midtrans dapat memperbarui status pembayaran secara otomatis pada database sistem.

5. Memastikan bahwa proses provisioning coin pada server Minecraft melalui Pterodactyl API dapat dijalankan secara otomatis setelah pembayaran berhasil.

6. Memastikan bahwa seluruh modul utama sistem dapat saling terhubung tanpa mengalami kehilangan data maupun kegagalan proses selama sistem digunakan.

2. Daftar Test Case

No	Nama Test Case	Deskripsi	Modul Terlibat	Output yang Diharapkan	Status
1	testLoginIntegrationSuccess()	Menguji proses login administrator menggunakan username dan password yang valid.	Authentication Module, Admin Controller, Database MySQL, Session Management.	Administrator berhasil login dan sesi admin berhasil dibuat.	Passed
2	testStoreIntegration()	Mengambil daftar produk aktif dari database dan menampilkannya pada halaman store.	Store Module, Product Controller, Product Model, Database MySQL.	Produk aktif berhasil ditampilkan dan produk tidak aktif tidak ditampilkan.	Passed
3	testPaymentIntegration()	Menguji proses checkout produk fixed coin melalui halaman purchase.	Purchase Module, Product Model, Checkout View, Database MySQL.	Halaman checkout berhasil ditampilkan sesuai produk yang dipilih.	Passed
4	testMidtransCallbackUpdatesPaymentStatus()	Menguji callback Midtrans dalam memperbarui status pembayaran transaksi.	Midtrans Callback Module, Transaction Model, Provision Module, Database MySQL.	Status pembayaran transaksi berhasil diperbarui menjadi success.	Passed
5	testProvisionCommandExecution()	Menguji eksekusi command provisioning coin setelah pembayaran berhasil.	Provision Module, Transaction Model, Provision Log Model, Pterodactyl API.	Provision log berhasil dibuat dan command pemberian coin berhasil dieksekusi.	Passed
6	testRetryProvisionIntegration()	Menguji proses retry provisioning pada transaksi yang sebelumnya gagal.	Provision Log Module, Admin Controller, Pterodactyl API, Database MySQL.	Status provision log berhasil diperbarui menjadi success setelah retry.	Passed
7	testTransactionHistoryIntegration()	Menguji pengambilan riwayat transaksi pengguna pada halaman store.	Transaction History Module, Transaction Model, Store Module, Database MySQL.	Riwayat transaksi pengguna berhasil ditampilkan.	Passed
8	testAdminTransactionMonitoring()	Menguji monitoring seluruh transaksi dan provision log oleh administrator.	Admin Dashboard Module, Transaction Model, Provision Log Model, Database MySQL.	Data transaksi dan log provisioning berhasil ditampilkan pada dashboard admin.	Passed
9	testCustomCoinPurchaseValidation()	Menguji validasi jumlah coin pada pembelian custom coin.	Purchase Module, Validation Module, Purchase Controller.	Input custom coin tidak valid berhasil ditolak oleh sistem.	Passed

3. Rangkuman Hasil

Total Test Case	: 9
Assertions	: 21
Lulus / Passed	: 9
Gagal	: 0
Persentase Keberhasilan	: 100%

4. Catatan Penting (Temuan Utama)

1. Seluruh proses integrasi antara website Laravel, database MySQL, Midtrans Payment Gateway, Mojang API, dan Pterodactyl API berhasil berjalan dengan baik tanpa ditemukan kegagalan proses.
2. Integrasi Midtrans berhasil melakukan sinkronisasi status pembayaran secara otomatis melalui callback notification.
3. Sistem provisioning coin menggunakan Pterodactyl API berhasil menjalankan command secara otomatis pada server Minecraft.
4. Proses komunikasi data antar modul berhasil menjaga konsistensi data selama proses login, transaksi pembayaran, provisioning coin, dan monitoring log berjalan.
5. Seluruh integration testing berhasil dijalankan tanpa ditemukan bug kritikal maupun kehilangan data selama proses pengujian dilakukan.

5. Rekomendasi

1. Disarankan untuk menambahkan pengujian terhadap kondisi ekstrem (*edge cases*) seperti kegagalan callback Midtrans, timeout API, maupun kegagalan eksekusi command pada Pterodactyl API.
2. Integration testing perlu dilakukan kembali apabila terdapat perubahan pada struktur database, endpoint API, maupun sistem provisioning server Minecraft.

3. Pengembangan selanjutnya dapat menambahkan stress testing dan load testing untuk mengukur performa sistem dalam menangani jumlah transaksi yang lebih besar.

4. Monitoring terhadap layanan Midtrans dan Pterodactyl API perlu dilakukan secara berkala untuk menjaga kestabilan integrasi sistem.

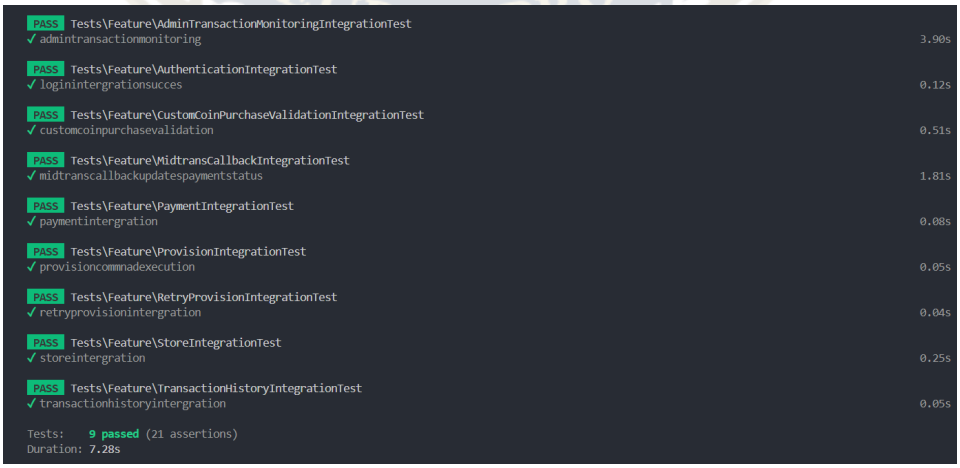
6. Contoh Kode Pengujian & Screenshot Log

Contoh Kode PHPUnit untuk Integration Testing

```
public function testloginintergrationsucces(): void
{
    $response = $this->post('/admin/login', [
        'username' => 'admin',
        'password' => 'admin123',
    ]);

    $response->assertRedirect('/admin/dashboard');
    $response->assertSessionHas('admin_logged_in', true);
}
```

Screenshot Log Pengujian



LAMPIRAN D

Acedmic Transcript

ACADEMIC TRANSCRIPT

Name	Julius Sebastian Wuwung
Student Identification Number	0706012210025
Place and Date of Birth	Surabaya / 2004-07-30
Faculty	School of Information Technology
Department	Informatics
Level	Bachelor

Code	Subject Name	Period	Credits	Grade	Grade Quality
IMT01303101	Algorithm & Programming	20221	5	C+	11.90
IMT01303102	Introduction to Information and Communication Technology	20221	2	B+	7.00
IMT01303103	Computer Organization and Architecture	20221	3	B+	10.50
IMT01303104	Linear Algebra	20221	3	A-	11.25
IMT01303201	Object-Oriented Programming	20222	4	C	8.00
IMT01303202	Web Programming	20222	4	C	8.00
IMT01303203	Database	20222	3	B	9.00
IMT01303204	Calculus	20222	2	B-	5.50
IMT01303301	Visual Programming	20231	5	C	10.00
IMT01303302	Web Development	20231	4	B	12.00
IMT01303303	Operating System	20231	3	B	9.00
IMT01303304	Discrete Mathematics	20231	3	B+	10.50
IMT01303401	Mobile Application Development	20232	5	B+	17.50
IMT01303402	Software Engineering	20232	3	C+	7.14
IMT01303403	Computer Networks	20232	3	B+	10.50
IMT01303404	Statistics & Probability	20232	4	C	8.00
IMT01303501	Computer Graphics	20241	3	B+	10.50
IMT01303601	Research Methodology	20242	3	B	9.00
IMT01306603	Computer Vision	20242	3	C+	7.14

Code	Subject Name	Period	Credits	Grade	Grade Quality
IMT01306608	Microservices Development	20242	3	B-	8.25
IMT01306609	Cloud Computing	20241	3	A	12.00
IMT01306610	Software Testing	20242	3	B+	10.50
IMT01306617	Game Design	20242	2	A-	7.50
IMT01306622	Blockchain	20252	3	T	0.00
IMT01307501	Digital Entrepreneurship Product Development	20241	5	A	20.00
IMT01307502	Cyber Security	20241	3	B	9.00
IMT01307503	Artificial Intelligence	20241	3	C	6.00
IMT01308701	Internship: Professional Development	20251	9	B-	24.75
IMT01308702	Internship: Product Development	20251	9	B-	24.75
IMT01309801	Final Project	20252	8	T	0.00
IMT01310801	Micro-credential	20252	2	T	0.00
UNC00301001	Religion	20221	2	B-	5.50
UNC00301002	Pancasila	20222	2	A-	7.50
UNC00301003	Civics	20251	2	A-	7.50
UNC00301004	Bahasa Indonesia	20242	2	A	8.00
UNC00302001	Menjadi Indonesia	20222	3	A-	11.25
UNC00302004	Ethics for the Information Age	20231	3	A-	11.25
UNC00302016	Entrepreneurship Essentials	20221	3	B+	10.50
UNC00304005	Healthy Lifestyle	20242	3	B	9.00
UNC00304009	Leisure Wellbeing	20241	3	B	9.00
UNC00304030	Digital Entrepreneurship and Innovation	20232	3	A-	11.25

Grade:

A	: 4.00	B	: 3.00	C	: 2.00	T	: 0.00
A-	: 3.75	B-	: 2.75	D	: 1.00		
B+	: 3.50	C+	: 2.38	E	: 0.00		

Cum. Credits	Completed Cum. Credits	131
Cum. GPA	Completed Cum. GPA	3.02

Surabaya, May 21, 2026

Authorized by Office Of The Registrar